100 SORU PANDAS-SQL

İlk olarak kolaylık osun diye tüm tabloları dataframe olarak python kısmına kaydettim.

In [1]:
```python
import pandas as pd
import sqlite3

# Veritabanı bağlantısı
conn = sqlite3.connect('sqlite-sakila.db')

# KURAL: Sadece tabloları ham haliyle çekiyoruz
actor_df = pd.read_sql_query("SELECT * FROM actor", conn)
film_df = pd.read_sql_query("SELECT * FROM film", conn)
film_actor_df = pd.read_sql_query("SELECT * FROM film_actor", conn)
inventory_df = pd.read_sql_query("SELECT * FROM inventory", conn)
rental_df = pd.read_sql_query("SELECT * FROM rental", conn)
payment_df = pd.read_sql_query("SELECT * FROM payment", conn)
customer_df = pd.read_sql_query("SELECT * FROM customer", conn)
category_df = pd.read_sql_query("SELECT * FROM category", conn)
film_category_df = pd.read_sql_query("SELECT * FROM film_category", conn)
address_df = pd.read_sql_query("SELECT * FROM address", conn)
city_df = pd.read_sql_query("SELECT * FROM city", conn)
store_df = pd.read_sql_query("SELECT * FROM store", conn)
staff_df = pd.read_sql_query("SELECT * FROM staff", conn)

print("Tüm tablolar DataFrame olarak yüklendi.")
```

```
Tüm tablolar DataFrame olarak yüklendi.
```

1. Soru: Tüm verileri göster

   A. SQLite Sorgusu ve Cevabı

```sql
SELECT * FROM film;
```

	film_id	title	description	release_year	language_id	original_language_id	rental_duration	rental_
1	1	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad…	2006	1	NULL	6	0.99
2	2	ACE GOLDFINGER	A Astounding Epistle of a Database …	2006	1	NULL	3	4.99
3	3	ADAPTATION HOLES	A Astounding Reflection of a …	2006	1	NULL	7	2.99
4	4	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee …	2006	1	NULL	5	2.99
5	5	AFRICAN EGG	A Fast-Paced Documentary of a Pastry…	2006	1	NULL	6	2.99
6	6	AGENT TRUMAN	A Intrepid Panorama of a Robot And a…	2006	1	NULL	3	2.99
7	7	AIRPLANE SIERRA	A Touching Saga of a Hunter And a …	2006	1	NULL	6	4.99
8	8	AIRPORT POLLOCK	A Epic Tale of a Moose And a Girl wh…	2006	1	NULL	6	4.99
9	9	ALABAMA DEVIL	A Thoughtful Panorama of a Database …	2006	1	NULL	3	2.99
10	10	ALADDIN CALENDAR	A Action-Packed Tale of a Man And a …	2006	1	NULL	6	4.99
11	11	ALAMO VIDEOTAPE	A Boring Epistle of a Butler And a …	2006	1	NULL	6	0.99
12	12	ALASKA PHANTOM	A Fanciful Saga of a Hunter And a …	2006	1	NULL	6	0.99
13	13	ALI FOREVER	A Action-Packed Drama of a Dentist …	2006	1	NULL	4	4.99
14	14	ALICE FANTASIA	A Emotional Drama of a A Shark And a…	2006	1	NULL	6	0.99
15	15	ALIEN CENTER	A Brilliant Drama of a Cat And a Mad…	2006	1	NULL	5	2.99
16	16	ALLEY EVOLUTION	A Fast-Paced Drama of a Robot And a …	2006	1	NULL	6	2.99
17	17	ALONE TRIP	A Fast-Paced Character Study of a …	2006	1	NULL	3	0.99
18	18	ALTER VICTORY	A Thoughtful Drama of a Composer And…	2006	1	NULL	6	0.99
19	19	AMADEUS HOLY	A Emotional Display of a Pioneer And…	2006	1	NULL	6	0.99
20	20	AMELIE HELLFIGHTERS	A Boring Drama of a Woman And a …	2006	1	NULL	4	4.99

B. Pandas Sorgusu ve Cevabı

In [2]:
```
# 1.soru
print(rental_df)
```

```
       rental_id             rental_date  inventory_id  customer_id  \
0              1  2005-05-24 22:53:30.000           367          130
1              2  2005-05-24 22:54:33.000          1525          459
2              3  2005-05-24 23:03:39.000          1711          408
3              4  2005-05-24 23:04:41.000          2452          333
4              5  2005-05-24 23:05:21.000          2079          222
...          ...                      ...           ...          ...
16039      16045  2005-08-23 22:25:26.000           772           14
16040      16046  2005-08-23 22:26:47.000          4364           74
16041      16047  2005-08-23 22:42:48.000          2088          114
16042      16048  2005-08-23 22:43:07.000          2019          103
16043      16049  2005-08-23 22:50:12.000          2666          393

                  return_date  staff_id         last_update
0      2005-05-26 22:04:30.000         1  2021-03-06 15:53:41
1      2005-05-28 19:40:33.000         1  2021-03-06 15:53:41
2      2005-06-01 22:12:39.000         1  2021-03-06 15:53:41
3      2005-06-03 01:43:41.000         2  2021-03-06 15:53:41
4      2005-06-02 04:33:21.000         1  2021-03-06 15:53:41
...                        ...       ...                  ...
16039  2005-08-25 23:54:26.000         1  2021-03-06 15:55:57
16040  2005-08-27 18:02:47.000         2  2021-03-06 15:55:57
16041  2005-08-25 02:48:48.000         2  2021-03-06 15:55:57
16042  2005-08-31 21:33:07.000         1  2021-03-06 15:55:57
16043  2005-08-30 01:01:12.000         2  2021-03-06 15:55:57

[16044 rows x 7 columns]
```

2. Soru: Her filmdeki oyuncuları listele

A. SQLite Sorgusu ve Cevabı

```sql
SELECT film.title,
       actor.first_name,
       actor.last_name
FROM film
JOIN film_actor ON film.film_id = film_actor.film_id
JOIN actor ON film_actor.actor_id = actor.actor_id;
```

	title	first_name	last_name
1	ACADEMY DINOSAUR	PENELOPE	GUINESS
2	ANACONDA CONFESSIONS	PENELOPE	GUINESS
3	ANGELS LIFE	PENELOPE	GUINESS
4	BULWORTH COMMANDMENTS	PENELOPE	GUINESS
5	CHEAPER CLYDE	PENELOPE	GUINESS
6	COLOR PHILADELPHIA	PENELOPE	GUINESS
7	ELEPHANT TROJAN	PENELOPE	GUINESS
8	GLEAMING JAWBREAKER	PENELOPE	GUINESS
9	HUMAN GRAFFITI	PENELOPE	GUINESS
10	KING EVOLUTION	PENELOPE	GUINESS
11	LADY STAGE	PENELOPE	GUINESS
12	LANGUAGE COWBOY	PENELOPE	GUINESS
13	MULHOLLAND BEAST	PENELOPE	GUINESS
14	OKLAHOMA JUMANJI	PENELOPE	GUINESS
15	RULES HUMAN	PENELOPE	GUINESS
16	SPLASH GUMP	PENELOPE	GUINESS
17	VERTIGO NORTHWEST	PENELOPE	GUINESS
18	WESTWARD SEABISCUIT	PENELOPE	GUINESS

B. Pandas Sorgusu ve Cevabı

In [3]:
```python
# 2.Soru
merged_step1 = pd.merge(film_df, film_actor_df, on='film_id', how='inner')

final_merge = pd.merge(merged_step1, actor_df, on='actor_id', how='inner')

result_q2 = final_merge[['title', 'first_name', 'last_name']]
print(result_q2)
```

```
                    title first_name last_name
0          ACADEMY DINOSAUR   PENELOPE   GUINESS
1       ANACONDA CONFESSIONS   PENELOPE   GUINESS
2                 ANGELS LIFE   PENELOPE   GUINESS
3       BULWORTH COMMANDMENTS   PENELOPE   GUINESS
4              CHEAPER CLYDE   PENELOPE   GUINESS
...                      ...        ...       ...
5457           SCALAWAG DUCK  CHRISTIAN    NEESON
5458        SENSIBILITY REAR  CHRISTIAN    NEESON
5459     SPIRITED CASUALTIES  CHRISTIAN    NEESON
5460              SPLASH GUMP  CHRISTIAN    NEESON
5461      WORLD LEATHERNECKS  CHRISTIAN    NEESON

[5462 rows x 3 columns]
```

3. Her filmde kaç oyuncu oynadı?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title,
       COUNT(fa.actor_id) AS actor_count
FROM film AS f
JOIN film_actor AS fa
  ON f.film_id = fa.film_id
GROUP BY f.title;
```

	title	actor_count
1	ACADEMY DINOSAUR	10
2	ACE GOLDFINGER	4
3	ADAPTATION HOLES	5
4	AFFAIR PREJUDICE	5
5	AFRICAN EGG	5
6	AGENT TRUMAN	7
7	AIRPLANE SIERRA	5
8	AIRPORT POLLOCK	4
9	ALABAMA DEVIL	9
10	ALADDIN CALENDAR	8
11	ALAMO VIDEOTAPE	4
12	ALASKA PHANTOM	7
13	ALI FOREVER	5
14	ALICE FANTASIA	4
15	ALIEN CENTER	6
16	ALLEY EVOLUTION	5
17	ALONE TRIP	8
18	ALTER VICTORY	4

B. Pandas Sorgusu ve Sonucu

In [4]:
```python
# Soru 3
merged_film_actor = pd.merge(film_df, film_actor_df, on='film_id', how='inner')

result_q3 = merged_film_actor.groupby('title')['actor_id'].count().reset_index()
result_q3.columns = ['title', 'actor_count']
print(result_q3)
```

```
                title  actor_count
0     ACADEMY DINOSAUR           10
1       ACE GOLDFINGER            4
2     ADAPTATION HOLES            5
3     AFFAIR PREJUDICE            5
4          AFRICAN EGG            5
..                 ...          ...
992     YOUNG LANGUAGE            5
993         YOUTH KICK            5
994       ZHIVAGO CORE            6
995  ZOOLANDER FICTION            5
996          ZORRO ARK            3

[997 rows x 2 columns]
```

4. Her oyuncu kaç filmde oynadı?

A. SQLite Sorgusu ve Cevabı

```
1  SELECT a.first_name, a.last_name, COUNT(fa.film_id) as movie_count
2  FROM actor a
3  JOIN film_actor fa ON a.actor_id = fa.actor_id
4  GROUP BY a.first_name, a.last_name;
```

	first_name	last_name	movie_count
1	ADAM	GRANT	18
2	ADAM	HOPPER	22
3	AL	GARLAND	26
4	ALAN	DREYFUSS	27
5	ALBERT	JOHANSSON	33
6	ALBERT	NOLTE	31
7	ALEC	WAYNE	29
8	ANGELA	HUDSON	34
9	ANGELA	WITHERSPOON	35
10	ANGELINA	ASTAIRE	31
11	ANNE	CRONYN	27
12	AUDREY	BAILEY	27
13	AUDREY	OLIVIER	25
14	BELA	WALKEN	30
15	BEN	HARRIS	23
16	BEN	WILLIS	33
17	BETTE	NICHOLSON	20
18	BOB	FAWCETT	25

B. Pandas Sorgusu ve Cevabı

In [5]:
```
1  # Soru 4
2  merged_q4 = pd.merge(actor_df, film_actor_df, on='actor_id', how='inner')
3
4  result_q4 = merged_q4.groupby(['first_name', 'last_name'])['film_id'].count().reset_index()
5  result_q4.rename(columns={'film_id': 'movie_count'}, inplace=True)
6
7  print(result_q4)
```

```
    first_name  last_name  movie_count
0         ADAM      GRANT           18
1         ADAM     HOPPER           22
2           AL    GARLAND           26
3         ALAN   DREYFUSS           27
4       ALBERT  JOHANSSON           33
..         ...        ...          ...
194       WILL     WILSON           31
195    WILLIAM    HACKMAN           27
196      WOODY    HOFFMAN           31
197      WOODY      JOLIE           31
198       ZERO       CAGE           25

[199 rows x 3 columns]
```

5. Envanterde olmayan filmler var mı ve varsa kaç tane?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT COUNT(*)
FROM film
WHERE film_id NOT IN (SELECT film_id FROM inventory);
```

	COUNT(*)
1	42

B. Pandas Sorgusu ve Sonucu

In [7]:
```python
# Soru 5
inventory_ids = inventory_df['film_id'].unique()

not_in_inventory = film_df[~film_df['film_id'].isin(inventory_ids)]

print(f"Envanterde olmayan film sayısı: {len(not_in_inventory)}")
```

```
Envanterde olmayan film sayısı: 42
```

6. Kiralanabilir olan her filmin kaç kez kiralandığını ve toplam gelirlerini getirin

A. SQLite Sorgusu ve Sonucu

```
2   FROM film f
3   JOIN inventory i ON f.film_id = i.film_id
4   JOIN rental r ON i.inventory_id = r.inventory_id
5   JOIN payment p ON r.rental_id = p.rental_id
6   GROUP BY f.title;
```

	title	rental_count	total_revenue
1	ACADEMY DINOSAUR	23	36.77
2	ACE GOLDFINGER	7	52.93
3	ADAPTATION HOLES	12	37.88
4	AFFAIR PREJUDICE	23	91.77
5	AFRICAN EGG	12	51.88
6	AGENT TRUMAN	21	126.79
7	AIRPLANE SIERRA	15	82.85
8	AIRPORT POLLOCK	18	102.82
9	ALABAMA DEVIL	12	71.88
10	ALADDIN CALENDAR	23	131.77
11	ALAMO VIDEOTAPE	24	35.76
12	ALASKA PHANTOM	26	44.74
13	ALI FOREVER	9	54.91
14	ALIEN CENTER	22	90.78
15	ALLEY EVOLUTION	14	52.86
16	ALONE TRIP	18	62.82
17	ALTER VICTORY	22	32.78
18	AMADEUS HOLY	21	33.79
19	AMELIE HELLFIGHTERS	10	67.9
20	AMERICAN CIRCUS	22	167.78

B. Pandas Sorgusu ve Sonucu

In [10]:

```
# Soru 6
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
m2 = pd.merge(m1, r_cols, on='inventory_id')
final_join = pd.merge(m2, p_cols, on='rental_id')

result_q6 = final_join.groupby('title').agg(
    rental_count=('rental_id', 'count'),
    total_revenue=('amount', 'sum')
).reset_index()

print(result_q6)
```

```
                title  rental_count  total_revenue
0     ACADEMY DINOSAUR            23          36.77
1       ACE GOLDFINGER             7          52.93
2     ADAPTATION HOLES            12          37.88
3     AFFAIR PREJUDICE            23          91.77
4          AFRICAN EGG            12          51.88
..                 ...           ...            ...
953    YOUNG LANGUAGE              7           6.93
954         YOUTH KICK             6          16.94
955       ZHIVAGO CORE             9          14.91
956  ZOOLANDER FICTION            17          73.83
957          ZORRO ARK            31         214.69

[958 rows x 3 columns]
```

7. Envanterde olmayan filmlerin kira oranlarını getirin

A. SQLite Sorgusu ve Sonucu

```
1  SELECT title, rental_rate
2  FROM film
3  WHERE film_id NOT IN (SELECT film_id FROM inventory);
```

	title	rental_rate
1	ALICE FANTASIA	0.99
2	APOLLO TEEN	2.99
3	ARGONAUTS TOWN	0.99
4	ARK RIDGEMONT	0.99
5	ARSENIC INDEPENDENCE	0.99
6	BOONDOCK BALLROOM	0.99
7	BUTCH PANTHER	0.99
8	CATCH AMISTAD	0.99
9	CHINATOWN GLADIATOR	4.99
10	CHOCOLATE DUCK	2.99
11	COMMANDMENTS EXPRESS	4.99
12	CROSSING DIVORCE	4.99
13	CROWDS TELEMARK	4.99
14	CRYSTAL BREAKING	2.99
15	DAZED PUNK	4.99
16	DELIVERANCE MULHOLLAND	0.99
17	FIREHOUSE VIETNAM	0.99
18	FLOATS GARDEN	2.99
19	FRANKENSTEIN STRANGER	0.99
20	GLADIATOR WESTWARD	4.99

B. Pandas Sorgusu ve Sonucu

In [12]:

```
# Soru 7
inventory_ids = inventory_df['film_id'].unique()
missing_films = film_df[~film_df['film_id'].isin(inventory_ids)]

result_q7 = missing_films[['title', 'rental_rate']]

print(result_q7)
```

```
                     title  rental_rate
13            ALICE FANTASIA         0.99
32               APOLLO TEEN         2.99
35            ARGONAUTS TOWN         0.99
37             ARK RIDGEMONT         0.99
40      ARSENIC INDEPENDENCE         0.99
86         BOONDOCK BALLROOM         0.99
107            BUTCH PANTHER         0.99
127            CATCH AMISTAD         0.99
143      CHINATOWN GLADIATOR         4.99
147           CHOCOLATE DUCK         2.99
170     COMMANDMENTS EXPRESS         4.99
191          CROSSING DIVORCE         4.99
194           CROWDS TELEMARK         4.99
197          CRYSTAL BREAKING         2.99
216                DAZED PUNK         4.99
220  DELIVERANCE MULHOLLAND         0.99
317        FIREHOUSE VIETNAM         0.99
324            FLOATS GARDEN         2.99
331    FRANKENSTEIN STRANGER         0.99
358       GLADIATOR WESTWARD         4.99
385                GUMP DATE         4.99
403            HATE HANDICAP         0.99
418              HOCUS FRIDA         2.99
494         KENTUCKIAN GIANT         2.99
496         KILL BROTHERHOOD         0.99
606              MUPPET MILE         4.99
641           ORDER BETRAYED         2.99
668            PEARL DESTINY         2.99
670          PERDITION FARGO         4.99
700            PSYCHO SHRUNK         2.99
711         RAIDERS ANTITRUST         0.99
712            RAINBOW SHOCK         4.99
741            ROOF CHAMPION         0.99
800            SISTER FREDDY         4.99
801              SKY MIRACLE         2.99
859         SUICIDES SILENCE         4.99
873              TADPOLE PARK         2.99
908         TREASURE COMMAND         0.99
942         VILLAIN DESPERATE         4.99
949              VOLUME HOUSE         4.99
953                 WAKE JAWS         4.99
954            WALLS ARTIST         4.99
```

8. Birden fazla DVD'yi iade etmeyen kaç müşteri var?

A. SQLite Sorgusu ve Sonucu

```
1  SELECT COUNT(DISTINCT customer_id)
2  FROM rental
3  WHERE return_date IS NULL
4  GROUP BY customer_id
5  HAVING COUNT(rental_id) > 1;
```

	COUNT(DISTINCT customer_id)
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1
14	1
15	1
16	1
17	1
18	1
19	1
20	1
21	1
22	1
23	1

B. Pandas Sorgusu ve Sonucu

In [13]:
```
1  # Soru 8
2  not_returned = rental_df[rental_df['return_date'].isnull()]
3
4  cust_counts = not_returned.groupby('customer_id').size()
5
6  count_q8 = len(cust_counts[cust_counts > 1])
7
8  print(f"Birden fazla DVD iade etmeyen müşteri sayısı: {count_q8}")
```

```
Birden fazla DVD iade etmeyen müşteri sayısı: 23
```

9. Her müşteri kaç film kiraladı?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, COUNT(r.rental_id)
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.first_name, c.last_name;
```

	first_name	last_name	COUNT(r.rental_id)
1	AARON	SELBY	24
2	ADAM	GOOCH	22
3	ADRIAN	CLARY	19
4	AGNES	BISHOP	23
5	ALAN	KAHN	26
6	ALBERT	CROUSE	23
7	ALBERTO	HENNING	21
8	ALEX	GRESHAM	33
9	ALEXANDER	FENNELL	36
10	ALFRED	CASILLAS	26
11	ALFREDO	MCADAMS	20
12	ALICE	STEWART	33
13	ALICIA	MILLS	21
14	ALLAN	CORNISH	19
15	ALLEN	BUTTERFIELD	21
16	ALLISON	STANLEY	27
17	ALMA	AUSTIN	35
18	ALVIN	DELOACH	29
19	AMANDA	CARTER	27
20	AMBER	DIXON	27

B. Pandas Sorgusu ve Sonucu

In [14]:
```python
# Soru 9
merged_q9 = pd.merge(customer_df, rental_df, on='customer_id')

result_q9 = merged_q9.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index()

print(result_q9)
```

```
    first_name last_name  rental_id
0        AARON     SELBY         24
1         ADAM     GOOCH         22
2       ADRIAN     CLARY         19
3        AGNES    BISHOP         23
4         ALAN      KAHN         26
..         ...       ...        ...
594     WILLIE   MARKHAM         25
595      WILMA  RICHARDS         20
596    YOLANDA    WEAVER         27
597     YVONNE   WATKINS         21
598    ZACHARY      HITE         31

[599 rows x 3 columns]
```

10. Türlerine göre en çok kiralanan filmler ve bunlara ne kadar ödendi?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name, f.title, COUNT(r.rental_id), SUM(p.amount)
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.name, f.title;
```

	name	title	COUNT(r.rental_id)	SUM(p.amount)
1	Action	AMADEUS HOLY	21	33.79
2	Action	AMERICAN CIRCUS	22	167.78
3	Action	ANTITRUST TOMATOES	10	37.9
4	Action	BAREFOOT MANCHURIAN	18	66.82
5	Action	BERETS AGENT	21	78.78
6	Action	BRIDE INTRIGUE	19	21.81
7	Action	BULL SHAWSHANK	16	21.84
8	Action	CADDYSHACK JEDI	16	51.84
9	Action	CAMPUS REMEMBER	19	90.81
10	Action	CASUALTIES ENCINO	9	72.91
11	Action	CELEBRITY HORN	24	32.76
12	Action	CLUELESS BUCKET	25	112.75
13	Action	CROW GREASE	12	18.88
14	Action	DANCES NONE	14	31.86
15	Action	DARKO DORADO	11	82.89
16	Action	DARN FORRESTER	18	93.82
17	Action	DEVIL DESIRE	15	83.85
18	Action	DRAGON SQUAD	11	27.89

B. Pandas Sorgusu ve Sonucu

In [20]:

```python
# Soru 10
f_cols = film_df[['film_id', 'title']]
fc_cols = film_category_df[['film_id', 'category_id']]
c_cols = category_df[['category_id', 'name']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(f_cols, fc_cols, on='film_id')
m2 = pd.merge(m1, c_cols, on='category_id')
m3 = pd.merge(m2, i_cols, on='film_id')
m4 = pd.merge(m3, r_cols, on='inventory_id')
final_q10 = pd.merge(m4, p_cols, on='rental_id')

result_q10 = final_q10.groupby(['name', 'title']).agg(
    rental_count=('rental_id', 'count'),
    total_revenue=('amount', 'sum')
).reset_index().sort_values(by='rental_count', ascending=False)

print(result_q10)
```

```
           name                title  rental_count  total_revenue
909      Travel   BUCKET BROTHERHOOD            34         180.66
536     Foreign     ROCKETEER MOTHER            33         116.67
757         New  RIDGEMONT SUBMARINE            32         130.68
572       Games       GRIT CLOCKWORK            32         110.68
93    Animation       JUGGLER HARDLY            32          96.68
..          ...                  ...           ...            ...
513     Foreign      INFORMER DOUBLE             5          27.95
570       Games          GLORY TRACY             5          14.95
656      Horror          TRAIN BUNCH             4          24.96
525     Foreign          MIXED DOORS             4          15.96
315  Documentary       HARDLY ROBBERS             4          15.96

[958 rows x 4 columns]
```

11.Tür ve Tarihe Göre Kiralama Sayısı ve Gelir

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name as genre, DATE(r.rental_date) as rental_date, COUNT(r.rental_id) as rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY c.name, DATE(r.rental_date)
ORDER BY rental_date;
```

	genre	rental_date	rental_count
1	Animation	2005-05-24	1
2	Children	2005-05-24	2
3	Comedy	2005-05-24	1
4	Family	2005-05-24	1
5	Horror	2005-05-24	2
6	Music	2005-05-24	1
7	Action	2005-05-25	10
8	Animation	2005-05-25	13
9	Children	2005-05-25	10
10	Classics	2005-05-25	9
11	Comedy	2005-05-25	7
12	Documentary	2005-05-25	10
13	Drama	2005-05-25	8
14	Family	2005-05-25	7
15	Foreign	2005-05-25	10
16	Games	2005-05-25	6
17	Horror	2005-05-25	3
18	Music	2005-05-25	8

B. Pandas Sorgusu ve Sonucu

In [21]:

```python
# Soru 11
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id', 'rental_date']]


m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, i_cols, on='film_id')
final_df = pd.merge(m2, r_cols, on='inventory_id')

final_df['rental_date'] = pd.to_datetime(final_df['rental_date'])
final_df['date_only'] = final_df['rental_date'].dt.date

result_q11 = final_df.groupby(['name', 'date_only'])['rental_id'].count().reset_index()
result_q11.columns = ['genre', 'rental_date', 'count']

print(result_q11)
```

```
      genre rental_date  count
0    Action  2005-05-25     10
1    Action  2005-05-26     15
2    Action  2005-05-27      9
3    Action  2005-05-28     20
4    Action  2005-05-29     12
..      ...         ...    ...
628  Travel  2005-08-20     29
629  Travel  2005-08-21     39
630  Travel  2005-08-22     21
631  Travel  2005-08-23     27
632  Travel  2006-02-14     10

[633 rows x 3 columns]
```

12.Kiralanabilir Filmler İçin Türlerine Göre Her Filmin Kaç Kez Kiralandığı

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name as genre, f.title, COUNT(r.rental_id) as rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY c.name, f.title
ORDER BY rental_count DESC;
```

	genre	title	rental_count
1	Travel	BUCKET BROTHERHOOD	34
2	Foreign	ROCKETEER MOTHER	33
3	Animation	JUGGLER HARDLY	32
4	Games	FORWARD TEMPLE	32
5	Games	GRIT CLOCKWORK	32
6	Music	SCALAWAG DUCK	32
7	New	RIDGEMONT SUBMARINE	32
8	Children	ROBBERS JOON	31
9	Classics	TIMBERLAND SKY	31
10	Comedy	ZORRO ARK	31
11	Documentary	WIFE TURN	31
12	Drama	HOBBIT ALIEN	31
13	Family	APACHE DIVINE	31
14	Family	NETWORK PEAK	31
15	Family	RUSH GOODFELLAS	31
16	Sci-Fi	GOODFELLAS SALUTE	31
17	Action	RUGRATS SHAKESPEARE	30
18	Action	SUSPECTS QUILLS	30

B. Pandas Sorgusu ve Sonucu

In [22]:

```python
# Soru 12
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, f_cols, on='film_id')
m3 = pd.merge(m2, i_cols, on='film_id')
final_df = pd.merge(m3, r_cols, on='inventory_id')

result_q12 = final_df.groupby(['name', 'title'])['rental_id'].count().reset_index()
result_q12.columns = ['genre', 'film_title', 'rental_count']

result_q12 = result_q12.sort_values(by='rental_count', ascending=False)

print(result_q12)
```

```
          genre           film_title  rental_count
909       Travel   BUCKET BROTHERHOOD            34
536      Foreign     ROCKETEER MOTHER            33
757          New  RIDGEMONT SUBMARINE            32
572        Games       GRIT CLOCKWORK            32
93     Animation       JUGGLER HARDLY            32
..           ...                  ...           ...
513      Foreign      INFORMER DOUBLE             5
570        Games          GLORY TRACY             5
656       Horror          TRAIN BUNCH             4
525      Foreign          MIXED DOORS             4
315  Documentary       HARDLY ROBBERS             4

[958 rows x 3 columns]
```

13.En çok rafta bekleyen filmler

A. SQLite Sorgusu ve Sonucu

```sql
SELECT
    f.title,
    -- MySQL'deki DATEDIFF(NOW(), rental_date) yerine:
    CAST(julianday('now') - julianday(r.rental_date) AS INTEGER) AS days_on_shelf
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NULL
ORDER BY days_on_shelf DESC;
```

	title	days_on_shelf
1	ACADEMY DINOSAUR	7412
2	HYDE DOCTOR	7235
3	HUNGER ROOF	7235
4	FRISCO FORREST	7235
5	TITANS JERK	7235
6	CONNECTION MICROCOSMOS	7235
7	HAUNTED ANTITRUST	7235
8	BULL SHAWSHANK	7235
9	GHOST GROUNDHOG	7235
10	PEACH INNOCENT	7235
11	SUIT WALLS	7235
12	WOMEN DORADO	7235
13	GRAIL FRANKENSTEIN	7235
14	ZHIVAGO CORE	7235
15	SONS INTERVIEW	7235
16	ENOUGH RAGING	7235
17	TITANIC BOONDOCK	7235
18	GUNFIGHT MOON	7235

B. Pandas Sorgusu ve Sonucu

In [24]:

```python
# Soru 13
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
# rental_date lazım (hesap için), return_date lazım (filtre için)
r_cols = rental_df[['inventory_id', 'rental_date', 'return_date']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
final_df = pd.merge(m1, r_cols, on='inventory_id')

on_rent = final_df[final_df['return_date'].isnull()].copy()

now = pd.Timestamp.now()
on_rent['rental_date'] = pd.to_datetime(on_rent['rental_date'])
on_rent['days_on_shelf'] = (now - on_rent['rental_date']).dt.days

result_q13 = on_rent[['title', 'days_on_shelf']].sort_values(by='days_on_shelf', ascending=False)

print(result_q13)
```

```
                      title  days_on_shelf
16          ACADEMY DINOSAUR           7413
9691         MUMMY CREATURES           7235
10016           NONE SPIKING           7235
10187    OPERATION OPERATION           7235
10223     OPPOSITE NECKLACE            7235
...                      ...            ...
4783            FAMILY SWEET           7235
5029        FIGHT JAWBREAKER           7235
5210             FLYING HOOK           7235
5291   FORRESTER COMANCHEROS           7235
15995           ZHIVAGO CORE           7235

[183 rows x 2 columns]
```

14.Geç, Erken ve Zamanında İade Edilen Kiralanmış Filmler

A. SQLite Sorgusu ve Sonucu

```sql
SELECT

    SUM(CASE WHEN julianday(r.return_date) > julianday(r.rental_date, '+' || f.rental_duration || ' days') THEN 1 ELSE 0 END) AS late_returns

    SUM(CASE WHEN julianday(r.return_date) < julianday(r.rental_date, '+' || f.rental_duration || ' days') THEN 1 ELSE 0 END) AS early_return

    SUM(CASE WHEN julianday(r.return_date) = julianday(r.rental_date, '+' || f.rental_duration || ' days') THEN 1 ELSE 0 END) AS on_time_retu
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
WHERE r.return_date IS NOT NULL;
```

	late_returns	early_returns	on_time_returns
1	8121	7738	2

B. Pandas Sorgusu ve Sonucu

In [25]:

```python
# Soru 14
f_cols = film_df[['film_id', 'rental_duration']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id', 'rental_date', 'return_date']]

m1 = pd.merge(r_cols, i_cols, on='inventory_id')
final_df = pd.merge(m1, f_cols, on='film_id')

final_df['rental_date'] = pd.to_datetime(final_df['rental_date'])
final_df['return_date'] = pd.to_datetime(final_df['return_date'])

final_df['due_date'] = final_df['rental_date'] + pd.to_timedelta(final_df['rental_duration'], unit='D')

def get_status(row):
    if pd.isnull(row['return_date']):
        return 'Not Returned'
    elif row['return_date'] > row['due_date']:
        return 'Late'
    elif row['return_date'] < row['due_date']:
        return 'Early'
    else:
        return 'On Time'

final_df['status'] = final_df.apply(get_status, axis=1)

result_q14 = final_df['status'].value_counts()

print(result_q14)
```

```
status
Late            8121
Early           7738
Not Returned     183
On Time            2
Name: count, dtype: int64
```

15.Hangi müşteri en çok DVD kiralamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, COUNT(r.rental_id) as rental_count
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.customer_id
ORDER BY rental_count DESC
LIMIT 1;
```

	first_name	last_name	rental_count
1	ELEANOR	HUNT	46

B. Pandas Sorgusu ve Sonucu

In [26]:

```python
# Soru 15
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]

merged_df = pd.merge(c_cols, r_cols, on='customer_id')

counts = merged_df.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index()
counts.rename(columns={'rental_id': 'rental_count'}, inplace=True)

top_customer = counts.sort_values(by='rental_count', ascending=False).iloc[0]

print(f"En çok kiralayan müşteri: {top_customer['first_name']} {top_customer['last_name']} ({top_customer['rental_count']} a
```

```
En çok kiralayan müşteri: ELEANOR HUNT (46 adet)
```

16.En popüler film kategorisi nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name, COUNT(r.rental_id) as rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY c.name
ORDER BY rental_count DESC
LIMIT 1;
```

	name	rental_count
1	Sports	1179

B. Pandas Sorgusu ve Sonucu

In [17]:

```python
# Soru 16
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, i_cols, on='film_id')
merged_df = pd.merge(m2, r_cols, on='inventory_id')

res_16 = merged_df.groupby('name')['rental_id'].count().reset_index()
res_16 = res_16.sort_values('rental_id', ascending=False).head(1)
print(res_16)
```

```
     name  rental_id
14  Sports       1179
```

17.Hangi çalışan en çok kiralama işlemi gerçekleştirmiş?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.first_name, s.last_name, COUNT(r.rental_id) as rental_count
FROM staff s
JOIN rental r ON s.staff_id = r.staff_id
GROUP BY s.staff_id
ORDER BY rental_count DESC
LIMIT 1;
```

	first_name	last_name	rental_count
1	Mike	Hillyer	8040

B. Pandas Sorgusu ve Sonucu

In [18]:
```python
# Soru 17
s_cols = staff_df[['staff_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'staff_id']]

merged_df = pd.merge(s_cols, r_cols, on='staff_id')

res_17 = merged_df.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index()
res_17 = res_17.sort_values('rental_id', ascending=False).head(1)
print(res_17)
```

```
  first_name last_name  rental_id
1       Mike   Hillyer       8040
```

18.En çok geliri hangi film getirmiş?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, SUM(p.amount) as total_revenue
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY f.title
ORDER BY total_revenue DESC
LIMIT 1;
```

	title	total_revenue
1	TELEGRAPH VOYAGE	231.73

B. Pandas Sorgusu ve Sonucu

In [19]:

```python
# Soru 18
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
m2 = pd.merge(m1, r_cols, on='inventory_id')
merged_df = pd.merge(m2, p_cols, on='rental_id')

res_18 = merged_df.groupby('title')['amount'].sum().reset_index()
res_18 = res_18.sort_values('amount', ascending=False).head(1)
print(res_18)
```

```
                title  amount
841  TELEGRAPH VOYAGE  231.73
```

19.Her müşteri için toplam harcama miktarını bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, SUM(p.amount) as total_spent
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id
ORDER BY total_spent DESC;
```

	first_name	last_name	total_spent
1	KARL	SEAL	221.55
2	ELEANOR	HUNT	216.54
3	CLARA	SHAW	195.58
4	RHONDA	KENNEDY	194.61
5	MARION	SNYDER	194.61
6	TOMMY	COLLAZO	186.62
7	WESLEY	BULL	177.6
8	TIM	CARY	175.61
9	MARCIA	DEAN	175.58
10	ANA	BRADLEY	174.66
11	JUNE	CARROLL	173.63
12	LENA	JENSEN	170.67
13	DIANE	COLLINS	169.65
14	ARNOLD	HAVENS	167.67
15	CURTIS	IRBY	167.62
16	MIKE	WAY	166.65
17	DAISY	BATES	162.62
18	TONYA	CHAPMAN	161.68
19	LOUIS	LEONE	161.65
20	GORDON	ALLARD	160.68

B. Pandas Sorgusu ve Sonucu

In [20]:
```python
# Soru 19
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
p_cols = payment_df[['customer_id', 'amount']]

merged_df = pd.merge(c_cols, p_cols, on='customer_id')

res_19 = merged_df.groupby(['first_name', 'last_name'])['amount'].sum().reset_index()
res_19 = res_19.sort_values('amount', ascending=False)
print(res_19.head)
```

```
<bound method NDFrame.head of     first_name last_name  amount
318       KARL      SEAL  221.55
175    ELEANOR      HUNT  216.54
105      CLARA      SHAW  195.58
474     RHONDA   KENNEDY  194.61
389     MARION    SNYDER  194.61
..         ...       ...     ...
33       ANNIE   RUSSELL   58.82
296     JOHNNY    TURPIN   57.81
71       BRIAN     WYMAN   52.88
351      LEONA    OBRIEN   50.86
83    CAROLINE    BOWMAN   50.85

[599 rows x 3 columns]>
```

20.Her kategorideki toplam kiralama sayısını ve gelirleri bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name, COUNT(r.rental_id) as rental_count, SUM(p.amount) as revenue
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.name;
```

	name	rental_count	revenue
1	Action	1112	4375.85
2	Animation	1166	4656.3
3	Children	945	3655.55
4	Classics	939	3639.59
5	Comedy	941	4383.58
6	Documentary	1050	4217.52
7	Drama	1060	4587.39
8	Family	1096	4226.07
9	Foreign	1033	4270.67
10	Games	969	4281.33
11	Horror	846	3722.54
12	Music	830	3417.72
13	New	940	4351.62
14	Sci-Fi	1101	4756.98
15	Sports	1179	5314.21
16	Travel	837	3549.64

B. Pandas Sorgusu ve Sonucu

In [21]:

```python
# Soru 20
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, i_cols, on='film_id')
m3 = pd.merge(m2, r_cols, on='inventory_id')
merged_df = pd.merge(m3, p_cols, on='rental_id')

res_20 = merged_df.groupby('name').agg(
    rental_count=('rental_id', 'count'),
    revenue=('amount', 'sum')
).reset_index()
print(res_20)
```

```
          name  rental_count  revenue
0       Action          1112  4375.85
1    Animation          1166  4656.30
2     Children           945  3655.55
3     Classics           939  3639.59
4       Comedy           941  4383.58
5  Documentary          1050  4217.52
6        Drama          1060  4587.39
7       Family          1096  4226.07
8      Foreign          1033  4270.67
9        Games           969  4281.33
10      Horror           846  3722.54
11       Music           830  3417.72
12         New           940  4351.62
13      Sci-Fi          1101  4756.98
14      Sports          1179  5314.21
15      Travel           837  3549.64
```

21.En uzun süre kirada kalmış filmleri bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, MAX(julianday(r.return_date) - julianday(r.rental_date)) as duration
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.title
ORDER BY duration DESC
LIMIT 5;
```

	title	duration
1	HOLOCAUST HIGHBALL	9.24930555559695
2	HIGHBALL POTTER	9.24930555513129
3	TRAMP OTHERS	9.2486111111939
4	PANIC CLUB	9.2486111111939
5	CONEHEADS SMOOCHY	9.2486111111939

B. Pandas Sorgusu ve Sonucu

In [22]:

```python
# Soru 21
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id', 'rental_date', 'return_date']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
merged_df = pd.merge(m1, r_cols, on='inventory_id')

merged_df['rental_date'] = pd.to_datetime(merged_df['rental_date'])
merged_df['return_date'] = pd.to_datetime(merged_df['return_date'])
merged_df['duration'] = (merged_df['return_date'] - merged_df['rental_date']).dt.total_seconds() / (24*3600)

res_21 = merged_df.groupby('title')['duration'].max().reset_index()
res_21 = res_21.sort_values('duration', ascending=False).head(5)
print(res_21)
```

```
                title  duration
400  HOLOCAUST HIGHBALL  9.249306
393     HIGHBALL POTTER  9.249306
160   CONEHEADS SMOOCHY  9.248611
625          PANIC CLUB  9.248611
535          MASK PEACH  9.248611
```

22.En az kiralanan 5 film hangileridir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, COUNT(r.rental_id) as rental_count
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY rental_count ASC
LIMIT 5;
```

	title	rental_count
1	HARDLY ROBBERS	4
2	MIXED DOORS	4
3	TRAIN BUNCH	4
4	BRAVEHEART HUMAN	5
5	BUNCH MINDS	5

B. Pandas Sorgusu ve Sonucu

In [23]:
```python
# Soru 22
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
merged_df = pd.merge(m1, r_cols, on='inventory_id')

res_22 = merged_df.groupby('title')['rental_id'].count().reset_index()
res_22 = res_22.sort_values('rental_id', ascending=True).head(5)
print(res_22)
```

```
                title  rental_id
558        MIXED DOORS          4
866        TRAIN BUNCH          4
378     HARDLY ROBBERS          4
585  MUSSOLINI SPOILERS          5
315   FREEDOM CLEOPATRA          5
```

23.

24.En fazla kazanç sağlayan 5 müşteriyi bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, COUNT(r.rental_id) as rental_count
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY rental_count ASC
LIMIT 5;SELECT c.first_name, c.last_name, SUM(p.amount) as total_spent
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id
ORDER BY total_spent DESC
LIMIT 5;
```

	first_name	last_name	total_spent
1	KARL	SEAL	221.55
2	ELEANOR	HUNT	216.54
3	CLARA	SHAW	195.58
4	RHONDA	KENNEDY	194.61
5	MARION	SNYDER	194.61

B. Pandas Sorgusu ve Sonucu

In [24]:

```python
# Soru 24
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
p_cols = payment_df[['customer_id', 'amount']]

merged_df = pd.merge(c_cols, p_cols, on='customer_id')

res_24 = merged_df.groupby(['first_name', 'last_name'])['amount'].sum().reset_index()
res_24 = res_24.sort_values('amount', ascending=False).head(5)
print(res_24)
```

```
    first_name last_name  amount
318       KARL      SEAL  221.55
175    ELEANOR      HUNT  216.54
105      CLARA      SHAW  195.58
474     RHONDA   KENNEDY  194.61
389     MARION    SNYDER  194.61
```

25.Her filmin ortalama kiralanma süresini bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, AVG(julianday(r.return_date) - julianday(r.rental_date)) as avg_duration
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.title;
```

	title	avg_duration
1	ACADEMY DINOSAUR	4.99712752523324
2	ACE GOLDFINGER	5.64270833336438
3	ADAPTATION HOLES	3.46562500010865
4	AFFAIR PREJUDICE	4.75833333336579
5	AFRICAN EGG	7.10662878774615
6	AGENT TRUMAN	5.87757936510302
7	AIRPLANE SIERRA	4.5819907406345
8	AIRPORT POLLOCK	5.05636574077006
9	ALABAMA DEVIL	5.57361111118613
10	ALADDIN CALENDAR	4.79924516914331
11	ALAMO VIDEOTAPE	5.01516203704523
12	ALASKA PHANTOM	5.29425747868103
13	ALI FOREVER	4.62812500004657
14	ALIEN CENTER	4.86038510105573
15	ALLEY EVOLUTION	5.5031250000133
16	ALONE TRIP	5.38619281039299
17	ALTER VICTORY	4.62989267670888
18	AMADEUS HOLY	4.92420138879679
19	AMELIE HELLFIGHTERS	4.87166666672565
20	AMERICAN CIRCUS	5.4205357142325

B. Pandas Sorgusu ve Sonucu

In [26]:

```python
# Soru 25
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id', 'rental_date', 'return_date']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
df = pd.merge(m1, r_cols, on='inventory_id')
df['rental_date'] = pd.to_datetime(df['rental_date'])
df['return_date'] = pd.to_datetime(df['return_date'])
df['duration'] = (df['return_date'] - df['rental_date']).dt.total_seconds() / (3600*24)

res_25 = df.groupby('title')['duration'].mean().reset_index()
print(res_25)
```

```
                title  duration
0     ACADEMY DINOSAUR  4.997128
1       ACE GOLDFINGER  5.642708
2     ADAPTATION HOLES  3.465625
3     AFFAIR PREJUDICE  4.758333
4          AFRICAN EGG  7.106629
..                 ...       ...
953     YOUNG LANGUAGE  4.624405
954         YOUTH KICK  5.562963
955       ZHIVAGO CORE  5.950087
956  ZOOLANDER FICTION  5.542647
957          ZORRO ARK  4.539180

[958 rows x 2 columns]
```

26.Her türde en popüler filmi bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT category_name, film_title, rental_count FROM (
    SELECT c.name as category_name, f.title as film_title, COUNT(r.rental_id) as rental_count,
    RANK() OVER (PARTITION BY c.name ORDER BY COUNT(r.rental_id) DESC) as rank
    FROM category c
    JOIN film_category fc ON c.category_id = fc.category_id
    JOIN film f ON fc.film_id = f.film_id
    JOIN inventory i ON f.film_id = i.film_id
    JOIN rental r ON i.inventory_id = r.inventory_id
    GROUP BY c.name, f.title
) WHERE rank = 1;
```

	category_name	film_title	rental_count
1	Action	SUSPECTS QUILLS	30
2	Action	RUGRATS SHAKESPEARE	30
3	Animation	JUGGLER HARDLY	32
4	Children	ROBBERS JOON	31
5	Classics	TIMBERLAND SKY	31
6	Comedy	ZORRO ARK	31
7	Documentary	WIFE TURN	31
8	Drama	HOBBIT ALIEN	31
9	Family	RUSH GOODFELLAS	31
10	Family	NETWORK PEAK	31
11	Family	APACHE DIVINE	31
12	Foreign	ROCKETEER MOTHER	33
13	Games	GRIT CLOCKWORK	32
14	Games	FORWARD TEMPLE	32
15	Horror	PULP BEVERLY	30
16	Music	SCALAWAG DUCK	32
17	New	RIDGEMONT SUBMARINE	32

B. Pandas Sorgusu ve Sonucu

In [27]:

```python
# Soru 26
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, f_cols, on='film_id')
m3 = pd.merge(m2, i_cols, on='film_id')
df = pd.merge(m3, r_cols, on='inventory_id')

counts = df.groupby(['name', 'title'])['rental_id'].count().reset_index(name='count')

res_26 = counts.groupby('name').apply(lambda x: x.nlargest(1, 'count')).reset_index(drop=True)
print(res_26)
```

```
          name                title  count
0       Action  RUGRATS SHAKESPEARE     30
1    Animation       JUGGLER HARDLY     32
2     Children         ROBBERS JOON     31
3     Classics       TIMBERLAND SKY     31
4       Comedy            ZORRO ARK     31
5  Documentary            WIFE TURN     31
6        Drama         HOBBIT ALIEN     31
7       Family        APACHE DIVINE     31
8      Foreign     ROCKETEER MOTHER     33
9        Games       FORWARD TEMPLE     32
10      Horror         PULP BEVERLY     30
11       Music        SCALAWAG DUCK     32
12         New  RIDGEMONT SUBMARINE     32
13      Sci-Fi    GOODFELLAS SALUTE     31
14      Sports  GLEAMING JAWBREAKER     29
15      Travel   BUCKET BROTHERHOOD     34
```

27.Her türde en fazla gelir sağlayan filmi bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name, f.title, SUM(p.amount) as total_revenue
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.name, f.title
ORDER BY total_revenue DESC
LIMIT 1;
```

	name	title	total_revenue
1	Music	TELEGRAPH VOYAGE	231.73

B. Pandas Sorgusu ve Sonucu

In [35]:

```python
# Soru 27
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, f_cols, on='film_id')
m3 = pd.merge(m2, i_cols, on='film_id')
m4 = pd.merge(m3, r_cols, on='inventory_id')
final_df = pd.merge(m4, p_cols, on='rental_id')

res_27 = final_df.groupby(['name', 'title'])['amount'].sum().reset_index()
res_27 = res_27.sort_values('amount', ascending=False).head(1)
print(res_27)
```

```
      name             title  amount
705  Music  TELEGRAPH VOYAGE  231.73
```

28.En çok DVD iade etmeyen müşteriyi bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, COUNT(r.rental_id) as unreturned_count
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
WHERE r.return_date IS NULL
GROUP BY c.customer_id
ORDER BY unreturned_count DESC
LIMIT 1;
```

	first_name	last_name	unreturned_count
1	TAMMY	SANDERS	3

B. Pandas Sorgusu ve Sonucu

In [28]:
```python
# Soru 28
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id', 'return_date']]

merged_df = pd.merge(c_cols, r_cols, on='customer_id')

# Filtreleme: Return date boş olanlar
unreturned = merged_df[merged_df['return_date'].isnull()]

res_28 = unreturned.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index()
res_28 = res_28.sort_values('rental_id', ascending=False).head(1)
print(res_28)
```

```
    first_name last_name  rental_id
144      TAMMY   SANDERS          3
```

29.En fazla kiralama yapan 5 çalışanı bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.first_name, s.last_name, COUNT(r.rental_id) as rental_count
FROM staff s
JOIN rental r ON s.staff_id = r.staff_id
GROUP BY s.staff_id
ORDER BY rental_count DESC
LIMIT 5;
```

	first_name	last_name	rental_count
1	Mike	Hillyer	8040
2	Jon	Stephens	8004

B. Pandas Sorgusu ve Sonucu

In [36]:
```python
# Soru 29
s_cols = staff_df[['staff_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'staff_id']]

merged_df = pd.merge(s_cols, r_cols, on='staff_id')

res_29 = merged_df.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index()
res_29 = res_29.sort_values('rental_id', ascending=False).head(5)
print(res_29)
```

```
  first_name last_name  rental_id
1       Mike   Hillyer       8040
0        Jon  Stephens       8004
```

30.En fazla kiralama yapan 5 müşteri hangi şubeden kiralama yapmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, s.store_id, COUNT(r.rental_id) as count
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN store s ON c.store_id = s.store_id
GROUP BY c.customer_id
ORDER BY count DESC
LIMIT 5;
```

	first_name	last_name	store_id	count
1	ELEANOR	HUNT	1	46
2	KARL	SEAL	2	45
3	CLARA	SHAW	1	42
4	MARCIA	DEAN	1	42
5	TAMMY	SANDERS	2	41

B. Pandas Sorgusu ve Sonucu

In [29]:
```python
# Soru 28
c_cols = customer_df[['customer_id', 'first_name', 'last_name', 'store_id']]
r_cols = rental_df[['rental_id', 'customer_id']]

merged_df = pd.merge(c_cols, r_cols, on='customer_id')

res_30 = merged_df.groupby(['first_name', 'last_name', 'store_id'])['rental_id'].count().reset_index()
res_30 = res_30.sort_values('rental_id', ascending=False).head(5)
print(res_30)
```

```
    first_name last_name  store_id  rental_id
175    ELEANOR      HUNT         1         46
318       KARL      SEAL         2         45
379     MARCIA      DEAN         1         42
105      CLARA      SHAW         1         42
536      TAMMY   SANDERS         2         41
```

31.Her türde en az kiralanan filmi bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name, f.title, COUNT(r.rental_id) as rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY c.name, f.title
ORDER BY rental_count ASC
LIMIT 1;
```

	name	title	rental_count
1	Documentary	HARDLY ROBBERS	4

B. Pandas Sorgusu ve Sonucu

```python
# Soru 31
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]


m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, f_cols, on='film_id')
m3 = pd.merge(m2, i_cols, on='film_id')
final_df = pd.merge(m3, r_cols, on='inventory_id')

res_31 = final_df.groupby(['name', 'title'])['rental_id'].count().reset_index(name='rental_count')
res_31 = res_31.sort_values('rental_count', ascending=True).head(1)
print(res_31)
```

```
          name           title  rental_count
315  Documentary  HARDLY ROBBERS             4
```

32.En çok kiralama yapan 5 müşteri hangi şehirde?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT city.city, customer.first_name, customer.last_name, COUNT(rental.rental_id) AS rental_count
FROM city
JOIN address ON city.city_id = address.city_id
JOIN customer ON address.address_id = customer.address_id
JOIN rental ON customer.customer_id = rental.customer_id
GROUP BY city.city, customer.first_name, customer.last_name
ORDER BY rental_count DESC
LIMIT 5;
```

	city	first_name	last_name	rental_count
1	Saint-Denis	ELEANOR	HUNT	46
2	Cape Coral	KARL	SEAL	45
3	Molodetno	CLARA	SHAW	42
4	Tanza	MARCIA	DEAN	42
5	Changhwa	TAMMY	SANDERS	41

B. Pandas Sorgusu ve Sonucu

```python
# Soru 32
city_cols = city_df[['city_id', 'city']]
addr_cols = address_df[['address_id', 'city_id']]
cust_cols = customer_df[['customer_id', 'address_id', 'first_name', 'last_name']]
rent_cols = rental_df[['rental_id', 'customer_id']]

m1 = pd.merge(city_cols, addr_cols, on='city_id')
m2 = pd.merge(m1, cust_cols, on='address_id')
final_df = pd.merge(m2, rent_cols, on='customer_id')

res_32 = final_df.groupby(['city', 'first_name', 'last_name'])['rental_id'].count().reset_index(name='rental_count')
res_32 = res_32.sort_values('rental_count', ascending=False).head(5)
print(res_32)
```

```
          city first_name last_name  rental_count
433  Saint-Denis    ELEANOR      HUNT            46
96    Cape Coral       KARL      SEAL            45
333    Molodetno      CLARA      SHAW            42
518        Tanza     MARCIA      DEAN            42
103     Changhwa      TAMMY   SANDERS            41
```

33.En çok kazanç sağlayan 5 müşteriyi hangi şehirde bulun?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT city.city, customer.first_name, customer.last_name, SUM(payment.amount) AS total_spent
FROM city
JOIN address ON city.city_id = address.city_id
JOIN customer ON address.address_id = customer.address_id
JOIN rental ON customer.customer_id = rental.customer_id
JOIN payment ON rental.rental_id = payment.rental_id
GROUP BY city.city, customer.first_name, customer.last_name
ORDER BY total_spent DESC
LIMIT 5;
```

	city	first_name	last_name	total_spent
1	Cape Coral	KARL	SEAL	221.55
2	Saint-Denis	ELEANOR	HUNT	216.54
3	Molodetno	CLARA	SHAW	195.58
4	Apeldoorn	RHONDA	KENNEDY	194.61
5	Santa Brbara dOeste	MARION	SNYDER	194.61

B. Pandas Sorgusu ve Sonucu

```python
# Soru 33
p_cols = payment_df[['rental_id', 'amount']]

full_df = pd.merge(final_df, p_cols, on='rental_id')

res_33 = full_df.groupby(['city', 'first_name', 'last_name'])['amount'].sum().reset_index(name='total_spent')
res_33 = res_33.sort_values('total_spent', ascending=False).head(5)
print(res_33)
```

```
                 city first_name last_name  total_spent
96           Cape Coral       KARL      SEAL       221.55
433         Saint-Denis    ELEANOR      HUNT       216.54
333           Molodetno      CLARA      SHAW       195.58
23            Apeldoorn     RHONDA   KENNEDY       194.61
447  Santa Brbara dOeste     MARION    SNYDER       194.61
```

34.En çok kiralanan 5 filmi hangi şehirde bulun?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT city.city, f.title, COUNT(r.rental_id) as rental_count
FROM city
JOIN address a ON city.city_id = a.city_id
JOIN customer c ON a.address_id = c.address_id
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
GROUP BY city.city, f.title
ORDER BY rental_count DESC
LIMIT 5;
```

	city	title	rental_count
1	Kurgan	DETECTIVE VISION	3
2	Lima	DISCIPLE MOTHER	3
3	Sorocaba	CADDYSHACK JEDI	3
4	Trshavn	FLATLINERS KILLER	3
5	Abu Dhabi	GENTLEMEN STAGE	2

B. Pandas Sorgusu ve Sonucu

In [30]:
```python
# Soru 34
city_cols = city_df[['city_id', 'city']]
addr_cols = address_df[['address_id', 'city_id']]
cust_cols = customer_df[['customer_id', 'address_id']]
rent_cols = rental_df[['rental_id', 'customer_id', 'inventory_id']]
inv_cols = inventory_df[['inventory_id', 'film_id']]
film_cols = film_df[['film_id', 'title']]

m1 = pd.merge(city_cols, addr_cols, on='city_id')
m2 = pd.merge(m1, cust_cols, on='address_id')
m3 = pd.merge(m2, rent_cols, on='customer_id')
m4 = pd.merge(m3, inv_cols, on='inventory_id')
df = pd.merge(m4, film_cols, on='film_id')

res_34 = df.groupby(['city', 'title'])['rental_id'].count().reset_index()
res_34 = res_34.sort_values('rental_id', ascending=False).head(5)
print(res_34)
```

```
          city              title  rental_id
14182   Trshavn  FLATLINERS KILLER          3
12735  Sorocaba    CADDYSHACK JEDI          3
7308     Kurgan   DETECTIVE VISION          3
7828       Lima    DISCIPLE MOTHER          3
2598    Caracas   AFFAIR PREJUDICE          2
```

35.En az kiralanan 5 filmi hangi şehirde bulun?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT city.city, film.title, COUNT(rental.rental_id) AS rental_count
FROM city
JOIN address ON city.city_id = address.city_id
JOIN customer ON address.address_id = customer.address_id
JOIN rental ON customer.customer_id = rental.customer_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
GROUP BY city.city, film.title
ORDER BY rental_count ASC
LIMIT 5;
```

	city	title	rental_count
1	A Corua (La Corua)	BLADE POLISH	1
2	A Corua (La Corua)	CAPER MOTIONS	1
3	A Corua (La Corua)	CHICKEN HELLFIGHTERS	1
4	A Corua (La Corua)	CHRISTMAS MOONSHINE	1
5	A Corua (La Corua)	CIDER DESIRE	1

B. Pandas Sorgusu ve Sonucu

```python
# Soru 35
i_cols = inventory_df[['inventory_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]
rent_cols_w_inv = rental_df[['rental_id', 'customer_id', 'inventory_id']]

m1 = pd.merge(city_df[['city_id','city']], address_df[['address_id','city_id']], on='city_id')
m2 = pd.merge(m1, customer_df[['customer_id','address_id']], on='address_id')
m3 = pd.merge(m2, rent_cols_w_inv, on='customer_id')
m4 = pd.merge(m3, i_cols, on='inventory_id')
complete_df = pd.merge(m4, f_cols, on='film_id')

res_35 = complete_df.groupby(['city', 'title'])['rental_id'].count().reset_index(name='rental_count')
res_35 = res_35.sort_values('rental_count', ascending=True).head(5)
print(res_35)
```

```
                  city           title  rental_count
0      A Corua (La Corua)     BLADE POLISH             1
10513          Phnom Penh  TOURIST PELICAN             1
10514          Phnom Penh  TWISTED PIRATES             1
10515          Phnom Penh   VOYAGE LEGALLY             1
10516          Phnom Penh        WIFE TURN             1
```

36.En çok kazanç sağlayan 5 filmi hangi şehirde bulun?

A. SQLite Sorgusu ve Sonucu

```
SELECT
    city.city,
    film.title,
    SUM(payment.amount) AS total_revenue
FROM city
JOIN address ON city.city_id = address.city_id
JOIN customer ON address.address_id = customer.address_id
JOIN rental ON customer.customer_id = rental.customer_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
JOIN payment ON rental.rental_id = payment.rental_id
GROUP BY city.city, film.title
ORDER BY total_revenue DESC
```

	city	title	total_revenue
1	Plock	CALIFORNIA BIRDS	18.98
2	Probolinggo	MINDS TRUMAN	18.98
3	Kamyin	ROSES TREASURE	17.98
4	Santa Brbara dOeste	WIFE TURN	17.98
5	Bijapur	WHISPERER GIANT	16.98

B. Pandas Sorgusu ve Sonucu

```
# Soru 36
city_cols = city_df[['city_id', 'city']]
addr_cols = address_df[['address_id', 'city_id']]
cust_cols = customer_df[['customer_id', 'address_id']]
rent_cols = rental_df[['rental_id', 'customer_id', 'inventory_id']]
inv_cols = inventory_df[['inventory_id', 'film_id']]
film_cols = film_df[['film_id', 'title']]
pay_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(city_cols, addr_cols, on='city_id')
m2 = pd.merge(m1, cust_cols, on='address_id')
m3 = pd.merge(m2, rent_cols, on='customer_id')
m4 = pd.merge(m3, inv_cols, on='inventory_id')
m5 = pd.merge(m4, film_cols, on='film_id')

final_df = pd.merge(m5, pay_cols, on='rental_id')

result_q36 = final_df.groupby(['city', 'title'])['amount'].sum().reset_index(name='total_revenue')

result_q36 = result_q36.sort_values(by='total_revenue', ascending=False).head(5)

print(result_q36)
```

```
                  city            title  total_revenue
10752         Probolinggo      MINDS TRUMAN          18.98
10565               Plock  CALIFORNIA BIRDS          18.98
6650               Kamyin    ROSES TREASURE          17.98
11854  Santa Brbara dOeste         WIFE TURN          17.98
3520                Datong      EAGLES PANKY          16.98
```

37.En az kazanç sağlayan 5 filmi hangi şehirde bulun?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT
    city.city,
    film.title,
    SUM(payment.amount) AS total_revenue
FROM city
JOIN address ON city.city_id = address.city_id
JOIN customer ON address.address_id = customer.address_id
JOIN rental ON customer.customer_id = rental.customer_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
JOIN payment ON rental.rental_id = payment.rental_id
GROUP BY city.city, film.title
ORDER BY total_revenue ASC
LIMIT 5;
```

	city	title	total_revenue
1	Allappuzha (Alleppey)	DEER VIRGINIAN	0
2	Aparecida de Goinia	WOMEN DORADO	0
3	Avellaneda	MOVIE SHAKESPEARE	0
4	Balurghat	CURTAIN VIDEOTAPE	0
5	Battambang	MINORITY KISS	0

B. Pandas Sorgusu ve Sonucu

```python
# Soru 37
res_37 = res_36.sort_values('total_revenue', ascending=True).head(5)
print(res_37)
```

```
                   city            title  total_revenue
3520             Datong     EAGLES PANKY          16.98
6650             Kamyin   ROSES TREASURE          17.98
11854  Santa Brbara dOeste        WIFE TURN          17.98
10752          Probolinggo     MINDS TRUMAN          18.98
10565                Plock  CALIFORNIA BIRDS          18.98
```

38.En fazla kiralama yapan müşteri hangi filmleri kiralamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, f.title
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
WHERE c.customer_id = (
    SELECT customer_id FROM rental
    GROUP BY customer_id
    ORDER BY COUNT(*) DESC LIMIT 1
);
```

	first_name	last_name	title
1	ELEANOR	HUNT	PREJUDICE OLEANDER
2	ELEANOR	HUNT	GUN BONNIE
3	ELEANOR	HUNT	SNATCHERS MONTEZUMA
4	ELEANOR	HUNT	ENGLISH BULWORTH
5	ELEANOR	HUNT	FORWARD TEMPLE
6	ELEANOR	HUNT	EGYPT TENENBAUMS
7	ELEANOR	HUNT	MAJESTIC FLOATS
8	ELEANOR	HUNT	FAMILY SWEET
9	ELEANOR	HUNT	ARIZONA BANG
10	ELEANOR	HUNT	REMEMBER DIARY
11	ELEANOR	HUNT	BIRD INDEPENDENCE
12	ELEANOR	HUNT	SECRET GROUNDHOG
13	ELEANOR	HUNT	MUSIC BOONDOCK
14	ELEANOR	HUNT	NAME DETECTIVE
15	ELEANOR	HUNT	AMERICAN CIRCUS
16	ELEANOR	HUNT	ARMAGEDDON LOST
17	ELEANOR	HUNT	MADIGAN DORADO

B. Pandas Sorgusu ve Sonucu

In [32]:

```python
# Soru 38
top_customer = rental_df.groupby('customer_id').size().sort_values(ascending=False).index[0]

cust_cols = customer_df[['customer_id', 'first_name', 'last_name']]
rent_cols = rental_df[['rental_id', 'customer_id', 'inventory_id']]
inv_cols = inventory_df[['inventory_id', 'film_id']]
film_cols = film_df[['film_id', 'title']]

target_rentals = rent_cols[rent_cols['customer_id'] == top_customer]

m1 = pd.merge(target_rentals, cust_cols, on='customer_id')
m2 = pd.merge(m1, inv_cols, on='inventory_id')
final_df = pd.merge(m2, film_cols, on='film_id')

res_38 = final_df[['first_name', 'last_name', 'title']]
print(res_38)
```

```
   first_name last_name              title
0     ELEANOR      HUNT   PREJUDICE OLEANDER
1     ELEANOR      HUNT          GUN BONNIE
2     ELEANOR      HUNT  SNATCHERS MONTEZUMA
3     ELEANOR      HUNT     ENGLISH BULWORTH
4     ELEANOR      HUNT       FORWARD TEMPLE
5     ELEANOR      HUNT     EGYPT TENENBAUMS
6     ELEANOR      HUNT      MAJESTIC FLOATS
7     ELEANOR      HUNT         FAMILY SWEET
8     ELEANOR      HUNT         ARIZONA BANG
9     ELEANOR      HUNT       REMEMBER DIARY
10    ELEANOR      HUNT    BIRD INDEPENDENCE
11    ELEANOR      HUNT     SECRET GROUNDHOG
12    ELEANOR      HUNT       MUSIC BOONDOCK
13    ELEANOR      HUNT       NAME DETECTIVE
14    ELEANOR      HUNT      AMERICAN CIRCUS
15    ELEANOR      HUNT      ARMAGEDDON LOST
16    ELEANOR      HUNT       MADIGAN DORADO
17    ELEANOR      HUNT           LADY STAGE
```

39.

40.En çok kazanç sağlayan müşteri hangi filmleri kiralamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, f.title
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
WHERE c.customer_id = (
    SELECT p.customer_id FROM payment p
    GROUP BY p.customer_id
    ORDER BY SUM(p.amount) DESC LIMIT 1
);
```

	first_name	last_name	title
1	KARL	SEAL	DESTINY SATURDAY
2	KARL	SEAL	CYCLONE FAMILY
3	KARL	SEAL	SLUMS DUCK
4	KARL	SEAL	FIDELITY DEVIL
5	KARL	SEAL	SPLASH GUMP
6	KARL	SEAL	MISSION ZOOLANDER
7	KARL	SEAL	MULHOLLAND BEAST
8	KARL	SEAL	PRINCESS GIANT
9	KARL	SEAL	PARIS WEEKEND
10	KARL	SEAL	RACER EGG
11	KARL	SEAL	WEDDING APOLLO
12	KARL	SEAL	BIKINI BORROWERS
13	KARL	SEAL	ROBBERY BRIGHT
14	KARL	SEAL	SPY MILE
15	KARL	SEAL	MOONWALKER FOOL
16	KARL	SEAL	METAL ARMAGEDDON
17	KARL	SEAL	RIDGEMONT SUBMARINE

B. Pandas Sorgusu ve Sonucu

In [33]:
```python
# Soru 40
top_paying_cust = payment_df.groupby('customer_id')['amount'].sum().sort_values(ascending=False).index[0]

rent_cols = rental_df[['rental_id', 'customer_id', 'inventory_id']]
inv_cols = inventory_df[['inventory_id', 'film_id']]
film_cols = film_df[['film_id', 'title']]

target_rentals = rent_cols[rent_cols['customer_id'] == top_paying_cust]

m1 = pd.merge(target_rentals, inv_cols, on='inventory_id')
res_40 = pd.merge(m1, film_cols, on='film_id')

print(res_40[['title']])
```

```
                 title
0       DESTINY SATURDAY
1         CYCLONE FAMILY
2             SLUMS DUCK
3         FIDELITY DEVIL
4            SPLASH GUMP
5      MISSION ZOOLANDER
6       MULHOLLAND BEAST
7         PRINCESS GIANT
8          PARIS WEEKEND
9              RACER EGG
10        WEDDING APOLLO
11        WEDDING APOLLO
12      BIKINI BORROWERS
13        ROBBERY BRIGHT
14             SPY MILE
15       MOONWALKER FOOL
16      METAL ARMAGEDDON
17    RIDGEMONT SUBMARINE
18           HIGH ENCINO
19          DURHAM PANKY
20           GANGS PRIDE
21            DATE SPEED
```

41.En az kazanç sağlayan müşteri hangi filmleri kiralamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT customer.first_name, customer.last_name, film.title, SUM(payment.amount) AS total_spent
FROM customer
JOIN rental ON customer.customer_id = rental.customer_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
JOIN payment ON rental.rental_id = payment.rental_id
WHERE customer.customer_id = (
    SELECT customer_id FROM payment GROUP BY customer_id ORDER BY SUM(amount) ASC LIMIT 1
)
GROUP BY customer.first_name, customer.last_name, film.title;
```

	first_name	last_name	title	total_spent
1	CAROLINE	BOWMAN	ARMAGEDDON LOST	0.99
2	CAROLINE	BOWMAN	CAMELOT VACATION	0.99
3	CAROLINE	BOWMAN	DISCIPLE MOTHER	5.99
4	CAROLINE	BOWMAN	EMPIRE MALKOVICH	0.99
5	CAROLINE	BOWMAN	FRENCH HOLIDAY	7.99
6	CAROLINE	BOWMAN	GOODFELLAS SALUTE	4.99
7	CAROLINE	BOWMAN	HOOSIERS BIRDCAGE	4.99
8	CAROLINE	BOWMAN	ILLUSION AMELIE	0.99
9	CAROLINE	BOWMAN	IMPOSSIBLE PREJUDICE	5.99
10	CAROLINE	BOWMAN	JADE BUNCH	2.99
11	CAROLINE	BOWMAN	KILLER INNOCENT	2.99
12	CAROLINE	BOWMAN	MASK PEACH	3.99
13	CAROLINE	BOWMAN	RESURRECTION SILVERADO	0.99
14	CAROLINE	BOWMAN	TEXAS WATCH	0.99
15	CAROLINE	BOWMAN	WHISPERER GIANT	4.99

B. Pandas Sorgusu ve Sonucu

```python
# Soru 41
low_customer_id = payment_df.groupby('customer_id')['amount'].sum().idxmin()

target_rentals = rental_df[rental_df['customer_id'] == low_customer_id][['inventory_id', 'customer_id']]

m1 = pd.merge(target_rentals, customer_df[['customer_id', 'first_name', 'last_name']], on='customer_id')
m2 = pd.merge(m1, inventory_df[['inventory_id', 'film_id']], on='inventory_id')
res_41 = pd.merge(m2, film_df[['film_id', 'title']], on='film_id')

print(res_41[['first_name', 'last_name', 'title']])
```

```
   first_name last_name                 title
0    CAROLINE    BOWMAN          FRENCH HOLIDAY
1    CAROLINE    BOWMAN         WHISPERER GIANT
2    CAROLINE    BOWMAN              MASK PEACH
3    CAROLINE    BOWMAN         ARMAGEDDON LOST
4    CAROLINE    BOWMAN         ILLUSION AMELIE
5    CAROLINE    BOWMAN       HOOSIERS BIRDCAGE
6    CAROLINE    BOWMAN        CAMELOT VACATION
7    CAROLINE    BOWMAN              JADE BUNCH
8    CAROLINE    BOWMAN         KILLER INNOCENT
9    CAROLINE    BOWMAN         DISCIPLE MOTHER
10   CAROLINE    BOWMAN       GOODFELLAS SALUTE
11   CAROLINE    BOWMAN        EMPIRE MALKOVICH
12   CAROLINE    BOWMAN  RESURRECTION SILVERADO
13   CAROLINE    BOWMAN             TEXAS WATCH
14   CAROLINE    BOWMAN    IMPOSSIBLE PREJUDICE
```

42.En az kazanç sağlayan müşteri hangi türde en fazla film kiralamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT customer.first_name, category.name AS genre, COUNT(rental.rental_id) AS rental_count
FROM customer
JOIN rental ON customer.customer_id = rental.customer_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
JOIN film_category ON film.film_id = film_category.film_id
JOIN category ON film_category.category_id = category.category_id
WHERE customer.customer_id = (
    SELECT customer_id FROM payment GROUP BY customer_id ORDER BY SUM(amount) ASC LIMIT 1
)
GROUP BY genre
ORDER BY rental_count DESC
LIMIT 1;
```

	first_name	genre	rental_count
1	CAROLINE	Sci-Fi	4

B. Pandas Sorgusu ve Sonucu

```python
# Soru 42
target_rentals = rental_df[rental_df['customer_id'] == low_customer_id][['inventory_id', 'customer_id']]

i_cols = inventory_df[['inventory_id', 'film_id']]
fc_cols = film_category_df[['film_id', 'category_id']]
cat_cols = category_df[['category_id', 'name']]
c_cols = customer_df[['customer_id', 'first_name']]

m1 = pd.merge(target_rentals, i_cols, on='inventory_id')
m2 = pd.merge(m1, fc_cols, on='film_id')
m3 = pd.merge(m2, cat_cols, on='category_id')
full_q42 = pd.merge(m3, c_cols, on='customer_id')

res_42 = full_q42.groupby(['first_name', 'name'])['inventory_id'].count().reset_index(name='count')
res_42 = res_42.sort_values('count', ascending=False).head(1)
print(res_42)
```

```
  first_name    name  count
6   CAROLINE  Sci-Fi      4
```

43.En çok kiralanan film hangi çalışan tarafından kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT staff.first_name, staff.last_name, film.title, COUNT(rental.rental_id) AS count
FROM staff
JOIN rental ON staff.staff_id = rental.staff_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
WHERE film.film_id = (
    SELECT i.film_id FROM rental r
    JOIN inventory i ON r.inventory_id = i.inventory_id
    GROUP BY i.film_id ORDER BY COUNT(*) DESC LIMIT 1
)
GROUP BY staff.first_name, staff.last_name, film.title;
```

	first_name	last_name	title	count
1	Jon	Stephens	BUCKET BROTHERHOOD	18
2	Mike	Hillyer	BUCKET BROTHERHOOD	16

B. Pandas Sorgusu ve Sonucu

```python
# Soru 43
rent_inv = pd.merge(rental_df[['rental_id', 'inventory_id']], inventory_df[['inventory_id', 'film_id']], on='inventory_id')
top_film_id = rent_inv['film_id'].value_counts().idxmax()
top_film_title = film_df[film_df['film_id'] == top_film_id]['title'].iloc[0]

target_inv_ids = inventory_df[inventory_df['film_id'] == top_film_id]['inventory_id']
target_rentals = rental_df[rental_df['inventory_id'].isin(target_inv_ids)]

res_43 = pd.merge(target_rentals, staff_df[['staff_id', 'first_name', 'last_name']], on='staff_id')
res_43 = res_43.groupby(['first_name', 'last_name']).size().reset_index(name='count')
res_43['film_title'] = top_film_title
print(res_43)
```

```
  first_name last_name  count          film_title
0        Jon  Stephens     18  BUCKET BROTHERHOOD
1       Mike   Hillyer     16  BUCKET BROTHERHOOD
```

44.En az kiralanan film hangi çalışan tarafından kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT staff.first_name, staff.last_name, film.title, COUNT(rental.rental_id) AS count
FROM staff
JOIN rental ON staff.staff_id = rental.staff_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
WHERE film.film_id = (
    SELECT i.film_id FROM rental r
    JOIN inventory i ON r.inventory_id = i.inventory_id
    GROUP BY i.film_id ORDER BY COUNT(*) ASC LIMIT 1
)
GROUP BY staff.first_name, staff.last_name, film.title;
```

	first_name	last_name	title	count
1	Jon	Stephens	HARDLY ROBBERS	3
2	Mike	Hillyer	HARDLY ROBBERS	1

B. Pandas Sorgusu ve Sonucu

```python
# Soru 44
least_film_id = rent_inv['film_id'].value_counts().idxmin()
least_film_title = film_df[film_df['film_id'] == least_film_id]['title'].iloc[0]

target_inv_ids = inventory_df[inventory_df['film_id'] == least_film_id]['inventory_id']
target_rentals = rental_df[rental_df['inventory_id'].isin(target_inv_ids)]

res_44 = pd.merge(target_rentals, staff_df[['staff_id', 'first_name', 'last_name']], on='staff_id')
res_44 = res_44.groupby(['first_name', 'last_name']).size().reset_index(name='count')
res_44['film_title'] = least_film_title
print(res_44)
```

```
  first_name last_name  count   film_title
0        Jon  Stephens      1  MIXED DOORS
1       Mike   Hillyer      3  MIXED DOORS
```

45.En çok kazanç sağlayan film hangi çalışan tarafından kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT staff.first_name, film.title, SUM(payment.amount) AS total_revenue
FROM staff
JOIN rental ON staff.staff_id = rental.staff_id
JOIN payment ON rental.rental_id = payment.rental_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
WHERE film.film_id = (
    SELECT i.film_id FROM payment p
    JOIN rental r ON p.rental_id = r.rental_id
    JOIN inventory i ON r.inventory_id = i.inventory_id
    GROUP BY i.film_id ORDER BY SUM(p.amount) DESC LIMIT 1
)
GROUP BY staff.first_name, film.title;
```

	first_name	title	total_revenue
1	Jon	TELEGRAPH VOYAGE	147.83
2	Mike	TELEGRAPH VOYAGE	83.9

B. Pandas Sorgusu ve Sonucu

```python
# Soru 46
m1 = pd.merge(payment_df[['rental_id', 'amount']], rental_df[['rental_id', 'inventory_id']], on='rental_id')
m2 = pd.merge(m1, inventory_df[['inventory_id', 'film_id']], on='inventory_id')
top_rev_film_id = m2.groupby('film_id')['amount'].sum().idxmax()

target_inv_ids = inventory_df[inventory_df['film_id'] == top_rev_film_id]['inventory_id']
target_rentals = rental_df[rental_df['inventory_id'].isin(target_inv_ids)]

m_staff = pd.merge(target_rentals, staff_df[['staff_id', 'first_name']], on='staff_id')
m_pay = pd.merge(m_staff, payment_df[['rental_id', 'amount']], on='rental_id')

res_45 = m_pay.groupby('first_name')['amount'].sum().reset_index(name='total_revenue')
print(res_45)
```

```
  first_name  total_revenue
0        Jon         147.83
1       Mike          83.90
```

46.En az kazanç sağlayan film hangi çalışan tarafından kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT staff.first_name, film.title, SUM(payment.amount) AS total_revenue
FROM staff
JOIN rental ON staff.staff_id = rental.staff_id
JOIN payment ON rental.rental_id = payment.rental_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
WHERE film.film_id = (
    SELECT i.film_id FROM payment p
    JOIN rental r ON p.rental_id = r.rental_id
    JOIN inventory i ON r.inventory_id = i.inventory_id
    GROUP BY i.film_id ORDER BY SUM(p.amount) ASC LIMIT 1
)
GROUP BY staff.first_name, film.title;
```

	first_name	title	total_revenue
1	Jon	OKLAHOMA JUMANJI	1.98
2	Mike	OKLAHOMA JUMANJI	3.96

B. Pandas Sorgusu ve Sonucu

```python
# Soru 46
low_rev_film_id = m2.groupby('film_id')['amount'].sum().idxmin()

target_inv_ids = inventory_df[inventory_df['film_id'] == low_rev_film_id]['inventory_id']
target_rentals = rental_df[rental_df['inventory_id'].isin(target_inv_ids)]

m_staff = pd.merge(target_rentals, staff_df[['staff_id', 'first_name']], on='staff_id')
m_pay = pd.merge(m_staff, payment_df[['rental_id', 'amount']], on='rental_id')

res_46 = m_pay.groupby('first_name')['amount'].sum().reset_index(name='total_revenue')
print(res_46)
```

```
  first_name  total_revenue
0        Jon           1.98
1       Mike           3.96
```

47.En çok kiralanan film hangi mağazada kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT store.store_id, film.title, COUNT(rental.rental_id) AS rental_count
FROM store
JOIN staff ON store.store_id = staff.store_id
JOIN rental ON staff.staff_id = rental.staff_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
WHERE film.film_id = (
    SELECT i.film_id FROM rental r JOIN inventory i ON r.inventory_id = i.inventory_id
    GROUP BY i.film_id ORDER BY COUNT(*) DESC LIMIT 1
)
GROUP BY store.store_id, film.title;
```

	store_id	title	rental_count
1	1	BUCKET BROTHERHOOD	16
2	2	BUCKET BROTHERHOOD	18

B. Pandas Sorgusu ve Sonucu

```python
# Soru 47
target_inv = inventory_df[inventory_df['film_id'] == top_film_id]

target_inv_ids = target_inv['inventory_id']
relevant_rentals = rental_df[rental_df['inventory_id'].isin(target_inv_ids)]

res_47 = pd.merge(relevant_rentals, inventory_df[['inventory_id', 'store_id']], on='inventory_id')
res_47 = res_47.groupby('store_id').size().reset_index(name='rental_count')
print(res_47)
```

```
   store_id  rental_count
0         1            17
1         2            17
```

48.En az kiralanan film hangi mağazada kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT store.store_id, film.title, COUNT(rental.rental_id) AS rental_count
FROM store
JOIN staff ON store.store_id = staff.store_id
JOIN rental ON staff.staff_id = rental.staff_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
WHERE film.film_id = (
    SELECT i.film_id FROM rental r JOIN inventory i ON r.inventory_id = i.inventory_id
    GROUP BY i.film_id ORDER BY COUNT(*) ASC LIMIT 1
)
GROUP BY store.store_id, film.title;
```

	store_id	title	rental_count
1	1	HARDLY ROBBERS	1
2	2	HARDLY ROBBERS	3

B. Pandas Sorgusu ve Sonucu

```python
# Soru 48
target_inv = inventory_df[inventory_df['film_id'] == least_film_id]
target_inv_ids = target_inv['inventory_id']

relevant_rentals = rental_df[rental_df['inventory_id'].isin(target_inv_ids)]

res_48 = pd.merge(relevant_rentals, inventory_df[['inventory_id', 'store_id']], on='inventory_id')
res_48 = res_48.groupby('store_id').size().reset_index(name='rental_count')
print(res_48)
```

```
   store_id  rental_count
0         1             4
```

49.En çok kazanç sağlayan film hangi mağazada kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT store.store_id, film.title, SUM(payment.amount) AS total_revenue
FROM store
JOIN staff ON store.store_id = staff.store_id
JOIN rental ON staff.staff_id = rental.staff_id
JOIN payment ON rental.rental_id = payment.rental_id
JOIN inventory ON rental.inventory_id = inventory.inventory_id
JOIN film ON inventory.film_id = film.film_id
WHERE film.film_id = (
    SELECT i.film_id FROM payment p JOIN rental r ON p.rental_id = r.rental_id JOIN inventory i ON r.inventory_id = i.inventory_id GROUP B
)
GROUP BY store.store_id, film.title;
```

	store_id	title	total_revenue
1	1	TELEGRAPH VOYAGE	83.9
2	2	TELEGRAPH VOYAGE	147.83

B. Pandas Sorgusu ve Sonucu

```python
# Soru 49
target_inv_ids = inventory_df[inventory_df['film_id'] == top_rev_film_id]['inventory_id']
target_rentals = rental_df[rental_df['inventory_id'].isin(target_inv_ids)]

m1 = pd.merge(target_rentals, inventory_df[['inventory_id', 'store_id']], on='inventory_id')
m2 = pd.merge(m1, payment_df[['rental_id', 'amount']], on='rental_id')

res_49 = m2.groupby('store_id')['amount'].sum().reset_index(name='total_revenue')
print(res_49)
```

```
   store_id  total_revenue
0         1         132.85
1         2          98.88
```

50.En az kazanç sağlayan film hangi mağazada kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, f.title, SUM(p.amount) as revenue
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
JOIN payment p ON r.rental_id = p.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
GROUP BY s.store_id, f.title
ORDER BY revenue ASC
LIMIT 1;
```

	store_id	title	revenue
1	1	FREEDOM CLEOPATRA	0.99

B. Pandas Sorgusu ve Sonucu

In [34]:

```python
# Soru 50
s_cols = store_df[['store_id']]
st_cols = staff_df[['staff_id', 'store_id']]
r_cols = rental_df[['rental_id', 'staff_id', 'inventory_id']]
p_cols = payment_df[['rental_id', 'amount']]
i_cols = inventory_df[['inventory_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
m2 = pd.merge(m1, r_cols, on='staff_id')
m3 = pd.merge(m2, p_cols, on='rental_id')
m4 = pd.merge(m3, i_cols, on='inventory_id')
df = pd.merge(m4, f_cols, on='film_id')

res_50 = df.groupby(['store_id', 'title'])['amount'].sum().reset_index()
res_50 = res_50.sort_values('amount', ascending=True).head(1)
print(res_50)
```

```
      store_id             title  amount
1112         2  COMANCHEROS ENEMY    0.99
```

51. Müşterilerin kiraladıkları filmlerin toplam kiralama süresi ne kadar?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name,
       SUM(julianday(r.return_date) - julianday(r.rental_date)) AS total_duration
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
WHERE r.return_date IS NOT NULL
GROUP BY c.first_name, c.last_name
ORDER BY total_duration DESC;
```

	first_name	last_name	total_duration
1	KARL	SEAL	264.151388887316
2	ELEANOR	HUNT	243.095138888806
3	CLARA	SHAW	235.04027777724
4	RHONDA	KENNEDY	229.187499998137
5	DAISY	BATES	221.306250001304
6	WESLEY	BULL	221.073611109518
7	TIM	CARY	218.062499998603
8	MARION	SNYDER	214.27916666586
9	JUNE	CARROLL	210.392361111939
10	MARCIA	DEAN	205.804166666232
11	BRANDON	HUEY	199.990277777892
12	GERTRUDE	CASTILLO	199.020833332092
13	ANA	BRADLEY	198.09861110989
14	CURTIS	IRBY	197.709722223226
15	SUE	PETERS	196.038194444031
16	NEIL	RENNER	195.032638889272
17	ARNOLD	HAVENS	194.854166667443
18	LOUIS	LEONE	193.85416666558

B. Pandas Sorgusu ve Sonucu

```python
# Soru 51
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id', 'rental_date', 'return_date']]

merged_df = pd.merge(c_cols, r_cols, on='customer_id')

merged_df['rental_date'] = pd.to_datetime(merged_df['rental_date'])
merged_df['return_date'] = pd.to_datetime(merged_df['return_date'])

merged_df = merged_df.dropna(subset=['return_date'])

merged_df['duration'] = (merged_df['return_date'] - merged_df['rental_date']).dt.total_seconds() / 86400

res_51 = merged_df.groupby(['first_name', 'last_name'])['duration'].sum().reset_index(name='total_duration')
res_51 = res_51.sort_values('total_duration', ascending=False)

print(res_51)
```

```
    first_name last_name  total_duration
318       KARL      SEAL      264.151389
175    ELEANOR      HUNT      243.095139
105      CLARA      SHAW      235.040278
474     RHONDA   KENNEDY      229.187500
125      DAISY     BATES      221.306250
..         ...       ...             ...
83    CAROLINE    BOWMAN       72.142361
296     JOHNNY    TURPIN       69.217361
33       ANNIE   RUSSELL       64.425694
71       BRIAN     WYMAN       59.231944
550    TIFFANY    JORDAN       59.197917

[599 rows x 3 columns]
```

52.En çok kiralanan türdeki filmler hangileri?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, f.title, COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY genre, f.title
ORDER BY rental_count DESC
LIMIT 1;
```

	genre	title	rental_count
1	Travel	BUCKET BROTHERHOOD	34

B. Pandas Sorgusu ve Sonucu

```python
# Soru 52
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, f_cols, on='film_id')
m3 = pd.merge(m2, i_cols, on='film_id')
final_df = pd.merge(m3, r_cols, on='inventory_id')

res_52 = final_df.groupby(['name', 'title'])['rental_id'].count().reset_index(name='rental_count')
res_52 = res_52.sort_values('rental_count', ascending=False).head(1)

print(res_52)
```

```
      name              title  rental_count
909  Travel  BUCKET BROTHERHOOD            34
```

53.En az kiralanan türdeki filmler hangileri?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, f.title, COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY genre, f.title
ORDER BY rental_count ASC
LIMIT 1;
```

	genre	title	rental_count
1	Documentary	HARDLY ROBBERS	4

B. Pandas Sorgusu ve Sonucu

```python
# Soru 53
res_53 = final_df.groupby(['name', 'title'])['rental_id'].count().reset_index(name='rental_count')
res_53 = res_53.sort_values('rental_count', ascending=True).head(1)

print(res_53)
```

```
          name           title  rental_count
315  Documentary  HARDLY ROBBERS             4
```

54.Her müşteri için toplam ödeme miktarını bulun.

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, SUM(p.amount) AS total_payment
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY total_payment DESC;
```

	first_name	last_name	total_payment
1	KARL	SEAL	221.55
2	ELEANOR	HUNT	216.54
3	CLARA	SHAW	195.58
4	MARION	SNYDER	194.61
5	RHONDA	KENNEDY	194.61
6	TOMMY	COLLAZO	186.62
7	WESLEY	BULL	177.6
8	TIM	CARY	175.61
9	MARCIA	DEAN	175.58
10	ANA	BRADLEY	174.66
11	JUNE	CARROLL	173.63
12	DIANE	COLLINS	169.65
13	LENA	JENSEN	168.68
14	ARNOLD	HAVENS	167.67
15	CURTIS	IRBY	167.62
16	MIKE	WAY	166.65
17	DAISY	BATES	162.62
18	TONYA	CHAPMAN	161.68
19	LOUIS	LEONE	161.65

B. Pandas Sorgusu ve Sonucu

```python
# Soru 54
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, p_cols, on='rental_id')

res_54 = final_df.groupby(['first_name', 'last_name'])['amount'].sum().reset_index(name='total_payment')
res_54 = res_54.sort_values('total_payment', ascending=False)

print(res_54)
```

```
    first_name last_name  total_payment
318       KARL      SEAL         221.55
175    ELEANOR      HUNT         216.54
105      CLARA      SHAW         195.58
474     RHONDA   KENNEDY         194.61
389     MARION    SNYDER         194.61
..         ...       ...            ...
33       ANNIE   RUSSELL          58.82
296     JOHNNY    TURPIN          57.81
71       BRIAN     WYMAN          52.88
351      LEONA    OBRIEN          50.86
83    CAROLINE    BOWMAN          50.85

[599 rows x 3 columns]
```

55.Hangi filmler en uzun süre kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, MAX(julianday(r.return_date) - julianday(r.rental_date)) AS max_rental_days
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.title
ORDER BY max_rental_days DESC
LIMIT 10;
```

	title	max_rental_days
1	HOLOCAUST HIGHBALL	9.24930555559695
2	HIGHBALL POTTER	9.24930555513129
3	TRAMP OTHERS	9.2486111111939
4	PANIC CLUB	9.2486111111939
5	CONEHEADS SMOOCHY	9.2486111111939
6	MASK PEACH	9.24861111072823
7	ATTACKS HATE	9.24722222238779
8	NOTORIOUS REUNION	9.24722222192213
9	TRACY CIDER	9.24652777798474
10	JERSEY SASSY	9.24652777798474

B. Pandas Sorgusu ve Sonucu

```python
#Soru 55
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id', 'rental_date', 'return_date']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
merged_df = pd.merge(m1, r_cols, on='inventory_id')

# Tarih dönüşümü
merged_df['rental_date'] = pd.to_datetime(merged_df['rental_date'])
merged_df['return_date'] = pd.to_datetime(merged_df['return_date'])

# Süre hesapla
merged_df['days'] = (merged_df['return_date'] - merged_df['rental_date']).dt.total_seconds() / 86400

res_55 = merged_df.groupby('title')['days'].max().reset_index(name='max_rental_days')
res_55 = res_55.sort_values('max_rental_days', ascending=False).head(10)

print(res_55)
```

```
                title  max_rental_days
400  HOLOCAUST HIGHBALL         9.249306
393     HIGHBALL POTTER         9.249306
160   CONEHEADS SMOOCHY         9.248611
625          PANIC CLUB         9.248611
535          MASK PEACH         9.248611
868        TRAMP OTHERS         9.248611
602   NOTORIOUS REUNION         9.247222
38         ATTACKS HATE         9.247222
863         TRACY CIDER         9.246528
461        JERSEY SASSY         9.246528
```

56.Hangi filmler en kısa süre kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, MIN(julianday(r.return_date) - julianday(r.rental_date)) AS min_rental_days
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.title
ORDER BY min_rental_days ASC
LIMIT 10;
```

	title	min_rental_days
1	CURTAIN VIDEOTAPE	0.75
2	DISCIPLE MOTHER	0.75
3	ROAD ROXANNE	0.750694444868714
4	HEAVEN FREEDOM	0.751388888806105
5	KISSING DOLLS	0.751388888806105
6	RACER EGG	0.751388888806105
7	RUGRATS SHAKESPEARE	0.751388888806105
8	FIDELITY DEVIL	0.751388889271766
9	STRAIGHT HOURS	0.751388889271766
10	MYSTIC TRUMAN	0.752083333209157

B. Pandas Sorgusu ve Sonucu

```python
# Soru 56
res_56 = merged_df.groupby('title')['days'].min().reset_index(name='min_rental_days')
res_56 = res_56.sort_values('min_rental_days', ascending=True).head(10)

print(res_56)
```

```
                title  min_rental_days
216     DISCIPLE MOTHER         0.750000
185   CURTAIN VIDEOTAPE         0.750000
701        ROAD ROXANNE         0.750694
815      STRAIGHT HOURS         0.751389
714  RUGRATS SHAKESPEARE         0.751389
296      FIDELITY DEVIL         0.751389
678           RACER EGG         0.751389
475       KISSING DOLLS         0.751389
387      HEAVEN FREEDOM         0.751389
748  SHAKESPEARE SADDLE         0.752083
```

57.Müşterilerin kiraladığı filmler için ortalama ödeme miktarını bulun

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, AVG(p.amount) AS avg_payment
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY avg_payment DESC;
```

	first_name	last_name	avg_payment
1	BRITTANY	RILEY	5.70428571428571
2	DON	BONE	5.35
3	KEVIN	SCHULER	5.30818181818182
4	LENA	JENSEN	5.27125
5	LONNIE	TIRADO	5.26777777777778
6	PAUL	TROUT	5.25086956521739
7	RUTH	MARTINEZ	5.24
8	LINDA	WILLIAMS	5.22076923076923
9	MIRIAM	MCKINNEY	5.22076923076923
10	LAURA	RODRIGUEZ	5.17181818181818
11	ANA	BRADLEY	5.13705882352941
12	MARSHALL	THORN	5.1204347826087
13	MAE	FLETCHER	5.11903225806452
14	MILTON	HOWLAND	5.11
15	ARNOLD	HAVENS	5.08090909090909
16	EDWIN	BURK	5.07695652173913
17	JOHNNIE	CHISHOLM	5.07333333333333
18	JEANNE	LAWSON	5.06407407407407

B. Pandas Sorgusu ve Sonucu

```python
# Soru 56
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, p_cols, on='rental_id')

res_57 = final_df.groupby(['first_name', 'last_name'])['amount'].mean().reset_index(name='avg_payment')
res_57 = res_57.sort_values('avg_payment', ascending=False)

print(res_57)
```

```
    first_name last_name  avg_payment
72    BRITTANY     RILEY     5.704286
154        DON      BONE     5.350000
331      KEVIN   SCHULER     5.308182
348       LENA    JENSEN     5.271250
364     LONNIE    TIRADO     5.267778
..         ...       ...          ...
589      WENDY  HARRISON     3.056667
309     JUDITH       COX     3.050606
62       BOBBY  BOUDREAU     3.047143
296     JOHNNY    TURPIN     3.042632
401     MATTIE   HOFFMAN     2.944545

[599 rows x 3 columns]
```

58.En çok kiralanan filmlerin ortalama kiralama süresi nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title,
       AVG(julianday(r.return_date) - julianday(r.rental_date)) AS avg_rental_days
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.title
ORDER BY COUNT(r.rental_id) DESC
LIMIT 10;
```

	title	avg_rental_days
1	BUCKET BROTHERHOOD	4.94844771242317
2	ROCKETEER MOTHER	5.38606902358658
3	SCALAWAG DUCK	4.06963975691178
4	GRIT CLOCKWORK	5.34086371524609
5	FORWARD TEMPLE	5.59524739578774
6	ZORRO ARK	4.53918010750485
7	WIFE TURN	4.792876344069
8	TIMBERLAND SKY	5.81135752690475
9	RUSH GOODFELLAS	4.48185483864959
10	ROBBERS JOON	4.86991487456966

B. Pandas Sorgusu ve Sonucu

```python
# Soru 58
res_58 = merged_df.groupby('title').agg(
    avg_rental_days=('days', 'mean'),
    rental_count=('rental_id', 'count')
).reset_index()

res_58 = res_58.sort_values('rental_count', ascending=False).head(10)

print(res_58[['title', 'avg_rental_days']])
```

```
                title  avg_rental_days
96    BUCKET BROTHERHOOD         4.948448
705     ROCKETEER MOTHER         5.386069
697  RIDGEMONT SUBMARINE         6.010909
361       GRIT CLOCKWORK         5.340864
465       JUGGLER HARDLY         5.509901
312       FORWARD TEMPLE         5.595247
733        SCALAWAG DUCK         4.069640
957            ZORRO ARK         4.539180
853       TIMBERLAND SKY         5.811358
29         APACHE DIVINE         4.269064
```

59.En az kiralanan filmlerin ortalama kiralama süresi nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title,
       AVG(julianday(r.return_date) - julianday(r.rental_date)) AS avg_rental_days
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.title
ORDER BY COUNT(r.rental_id) ASC
LIMIT 10;
```

	title	avg_rental_days
1	HARDLY ROBBERS	6.94843749993015
2	MIXED DOORS	5.95329861098435
3	TRAIN BUNCH	3.34930555545725
4	BRAVEHEART HUMAN	4.01986111123115
5	BUBBLE GROSSE	4.43083333326504
6	BUNCH MINDS	5.52444444447756
7	CONSPIRACY SPIRIT	4.00999999996275
8	FEVER EMPIRE	5.75569444419816
9	FREEDOM CLEOPATRA	2.95972222229466
10	FULL FLATLINERS	3.7673611111939

B. Pandas Sorgusu ve Sonucu

```python
# Soru 59
stats = merged_df.groupby('title').agg(
    avg_rental_days=('days', 'mean'),
    rental_count=('rental_id', 'count')
).reset_index()

res_59 = stats.sort_values('rental_count', ascending=True).head(10)

print(res_59[['title', 'avg_rental_days']])
```

```
                title  avg_rental_days
558         MIXED DOORS         5.953299
866         TRAIN BUNCH         3.349306
378      HARDLY ROBBERS         6.948437
585  MUSSOLINI SPOILERS         4.038611
315   FREEDOM CLEOPATRA         2.959722
669        PRIVATE DROP         4.949583
293        FEVER EMPIRE         5.755694
168   CONSPIRACY SPIRIT         4.010000
532     MANNEQUIN WORST         4.414167
435     INFORMER DOUBLE         3.394583
```

60.En çok kazanç sağlayan müşterilerin ortalama ödeme miktarı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, AVG(p.amount) AS avg_payment
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY SUM(p.amount) DESC
LIMIT 10;
```

	first_name	last_name	avg_payment
1	KARL	SEAL	4.92333333333333
2	ELEANOR	HUNT	4.70739130434783
3	CLARA	SHAW	4.65666666666667
4	MARION	SNYDER	4.99
5	RHONDA	KENNEDY	4.99
6	TOMMY	COLLAZO	4.91105263157895
7	WESLEY	BULL	4.44
8	TIM	CARY	4.50282051282051
9	MARCIA	DEAN	4.18047619047619
10	ANA	BRADLEY	5.13705882352941

B. Pandas Sorgusu ve Sonucu

```python
# Soru 60
cust_stats = final_df.groupby(['first_name', 'last_name']).agg(
    avg_payment=('amount', 'mean'),
    total_spent=('amount', 'sum')
).reset_index()

res_60 = cust_stats.sort_values('total_spent', ascending=False).head(10)

print(res_60[['first_name', 'last_name', 'avg_payment']])
```

```
    first_name last_name  avg_payment
318       KARL      SEAL     4.923333
175    ELEANOR      HUNT     4.707391
105      CLARA      SHAW     4.656667
474     RHONDA   KENNEDY     4.990000
389     MARION    SNYDER     4.990000
556      TOMMY   COLLAZO     4.911053
590     WESLEY      BULL     4.440000
551        TIM      CARY     4.502821
379     MARCIA      DEAN     4.180476
21         ANA   BRADLEY     5.137059
```

61.En az kazanç sağlayan müşterilerin ortalama ödeme miktarı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, AVG(p.amount) AS avg_payment
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY SUM(p.amount) ASC
LIMIT 10;
```

	first_name	last_name	avg_payment
1	CAROLINE	BOWMAN	3.39
2	LEONA	OBRIEN	3.63285714285714
3	BRIAN	WYMAN	4.40666666666667
4	JOHNNY	TURPIN	3.04263157894737
5	ANNIE	RUSSELL	3.26777777777778
6	KATHERINE	RIVERA	4.20428571428571
7	TIFFANY	JORDAN	4.27571428571429
8	ANITA	MORALES	4.19
9	MATTIE	HOFFMAN	2.94454545454545
10	KIRK	STCLAIR	3.41105263157895

B. Pandas Sorgusu ve Sonucu

```python
# Soru 61
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, p_cols, on='rental_id')

stats = final_df.groupby(['first_name', 'last_name']).agg(
    avg_payment=('amount', 'mean'),
    total_spent=('amount', 'sum')
).reset_index()

res_61 = stats.sort_values('total_spent', ascending=True).head(10)
print(res_61[['first_name', 'last_name', 'avg_payment']])
```

```
    first_name last_name  avg_payment
83    CAROLINE    BOWMAN     3.390000
351      LEONA    OBRIEN     3.632857
71       BRIAN     WYMAN     4.406667
296     JOHNNY    TURPIN     3.042632
33       ANNIE   RUSSELL     3.267778
319  KATHERINE     RIVERA     4.204286
550    TIFFANY    JORDAN     4.275714
28       ANITA   MORALES     4.190000
401     MATTIE   HOFFMAN     2.944545
334       KIRK   STCLAIR     3.411053
```

62.Mağazalardaki toplam kiralama süresini bulun.

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id,
       SUM(julianday(r.return_date) - julianday(r.rental_date)) AS total_rental_days
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
WHERE r.return_date IS NOT NULL
GROUP BY s.store_id;
```

	store_id	total_rental_days
1	1	39878.7555555534
2	2	39828.0090277721

B. Pandas Sorgusu ve Sonucu

```python
# Soru 62
s_cols = store_df[['store_id']]
st_cols = staff_df[['staff_id', 'store_id']]
r_cols = rental_df[['rental_id', 'staff_id', 'rental_date', 'return_date']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
merged_df = pd.merge(m1, r_cols, on='staff_id')
merged_df['rental_date'] = pd.to_datetime(merged_df['rental_date'])
merged_df['return_date'] = pd.to_datetime(merged_df['return_date'])
merged_df = merged_df.dropna(subset=['return_date'])
merged_df['duration'] = (merged_df['return_date'] - merged_df['rental_date']).dt.total_seconds() / 86400

res_62 = merged_df.groupby('store_id')['duration'].sum().reset_index(name='total_rental_days')
print(res_62)
```

```
   store_id  total_rental_days
0         1       39878.755556
1         2       39828.009028
```

63.En uzun süre kiralanan film hangi mağazada kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, MAX(julianday(r.return_date) - julianday(r.rental_date)) AS max_rental_days
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN staff s ON r.staff_id = s.staff_id -- Mağaza bilgisi staff üzerinden
WHERE r.return_date IS NOT NULL
GROUP BY f.title, s.store_id
ORDER BY max_rental_days DESC
LIMIT 1;
```

	title	max_rental_days
1	HOLOCAUST HIGHBALL	9.24930555559695

B. Pandas Sorgusu ve Sonucu

```python
# Soru 63
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id', 'staff_id', 'rental_date', 'return_date']]
st_cols = staff_df[['staff_id', 'store_id']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
m2 = pd.merge(m1, r_cols, on='inventory_id')
merged_df = pd.merge(m2, st_cols, on='staff_id')

merged_df['rental_date'] = pd.to_datetime(merged_df['rental_date'])
merged_df['return_date'] = pd.to_datetime(merged_df['return_date'])
merged_df['days'] = (merged_df['return_date'] - merged_df['rental_date']).dt.total_seconds() / 86400

res_63 = merged_df.sort_values('days', ascending=False).head(1)
print(res_63[['title', 'store_id', 'days']])
```

```
                title  store_id      days
11418  HOLOCAUST HIGHBALL         1  9.249306
```

64.En kısa süre kiralanan film hangi mağazada kiralanmış?

A. SQLite Sorgusu ve Sonucu

```
SELECT f.title, MIN(julianday(r.return_date) - julianday(r.rental_date)) AS min_rental_days FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN staff s ON r.staff_id = s.staff_id -- Mağaza bilgisi staff üzerinden
WHERE r.return_date IS NOT NULL
GROUP BY f.title, s.store_id
ORDER BY min_rental_days ASC
LIMIT 1;
```

	title	min_rental_days
1	CURTAIN VIDEOTAPE	0.75

B. Pandas Sorgusu ve Sonucu

```
# Soru 64
res_64 = merged_df.sort_values('days', ascending=True).head(1)
print(res_64[['title', 'store_id', 'days']])
```

```
                title  store_id  days
1546  CURTAIN VIDEOTAPE         2  0.75
```

65.Her filmin ortalama kiralama süresi nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, AVG(julianday(r.return_date) - julianday(r.rental_date)) AS avg_rental_days
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date IS NOT NULL
GROUP BY f.title;
```

	title	avg_rental_days
1	ACADEMY DINOSAUR	4.99712752523324
2	ACE GOLDFINGER	5.64270833336438
3	ADAPTATION HOLES	3.46562500010865
4	AFFAIR PREJUDICE	4.75833333336579
5	AFRICAN EGG	7.10662878774615
6	AGENT TRUMAN	5.87757936510302
7	AIRPLANE SIERRA	4.5819907406345
8	AIRPORT POLLOCK	5.05636574077006
9	ALABAMA DEVIL	5.57361111118613
10	ALADDIN CALENDAR	4.79924516914331
11	ALAMO VIDEOTAPE	5.01516203704523
12	ALASKA PHANTOM	5.29425747868103
13	ALI FOREVER	4.62812500004657
14	ALIEN CENTER	4.86038510105573
15	ALLEY EVOLUTION	5.5031250000133
16	ALONE TRIP	5.38619281039299
17	ALTER VICTORY	4.6288267670888

B. Pandas Sorgusu ve Sonucu

```python
# Soru 65
res_65 = merged_df.groupby('title')['days'].mean().reset_index(name='avg_rental_days')
print(res_65)
```

```
                title  avg_rental_days
0     ACADEMY DINOSAUR         4.997128
1       ACE GOLDFINGER         5.642708
2     ADAPTATION HOLES         3.465625
3     AFFAIR PREJUDICE         4.758333
4          AFRICAN EGG         7.106629
..                 ...              ...
953     YOUNG LANGUAGE         4.624405
954         YOUTH KICK         5.562963
955       ZHIVAGO CORE         5.950087
956  ZOOLANDER FICTION         5.542647
957          ZORRO ARK         4.539180

[958 rows x 2 columns]
```

66.Hangi filmler en çok kiralanan kategoride?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, f.title, COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY genre, f.title
ORDER BY rental_count DESC;
```

	genre	title	rental_count
1	Travel	BUCKET BROTHERHOOD	34
2	Foreign	ROCKETEER MOTHER	33
3	Animation	JUGGLER HARDLY	32
4	Games	FORWARD TEMPLE	32
5	Games	GRIT CLOCKWORK	32
6	Music	SCALAWAG DUCK	32
7	New	RIDGEMONT SUBMARINE	32
8	Children	ROBBERS JOON	31
9	Classics	TIMBERLAND SKY	31
10	Comedy	ZORRO ARK	31
11	Documentary	WIFE TURN	31
12	Drama	HOBBIT ALIEN	31
13	Family	APACHE DIVINE	31
14	Family	NETWORK PEAK	31
15	Family	RUSH GOODFELLAS	31
16	Sci-Fi	GOODFELLAS SALUTE	31
17	Action	RUGRATS SHAKESPEARE	30

B. Pandas Sorgusu ve Sonucu

```python
# Soru 66
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, f_cols, on='film_id')
m3 = pd.merge(m2, i_cols, on='film_id')
final_df = pd.merge(m3, r_cols, on='inventory_id')

res_66 = final_df.groupby(['name', 'title'])['rental_id'].count().reset_index(name='rental_count')
res_66 = res_66.sort_values('rental_count', ascending=False)

print(res_66)
```

```
           name                title  rental_count
909       Travel   BUCKET BROTHERHOOD            34
536      Foreign     ROCKETEER MOTHER            33
757          New  RIDGEMONT SUBMARINE            32
572        Games       GRIT CLOCKWORK            32
93     Animation       JUGGLER HARDLY            32
..           ...                  ...           ...
513      Foreign      INFORMER DOUBLE             5
570        Games          GLORY TRACY             5
656       Horror          TRAIN BUNCH             4
525      Foreign          MIXED DOORS             4
315  Documentary       HARDLY ROBBERS             4

[958 rows x 3 columns]
```

67.Hangi filmler en az kiralanan kategoride?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, f.title, COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY genre, f.title
ORDER BY rental_count ASC;
```

	genre	title	rental_count
1	Documentary	HARDLY ROBBERS	4
2	Foreign	MIXED DOORS	4
3	Horror	TRAIN BUNCH	4
4	Children	FULL FLATLINERS	5
5	Classics	CONSPIRACY SPIRIT	5
6	Comedy	FREEDOM CLEOPATRA	5
7	Documentary	HUNTER ALTER	5
8	Drama	BUNCH MINDS	5
9	Family	BRAVEHEART HUMAN	5
10	Foreign	INFORMER DOUBLE	5
11	Games	FEVER EMPIRE	5
12	Games	GLORY TRACY	5
13	Games	PRIVATE DROP	5
14	Games	SEVEN SWARM	5
15	New	MANNEQUIN WORST	5
16	Sports	MUSSOLINI SPOILERS	5
17	Travel	TRAFFIC HOBBIT	5

B. Pandas Sorgusu ve Sonucu

```python
# Soru 67
res_67 = res_66.sort_values('rental_count', ascending=True)
print(res_67)
```

```
            name                title  rental_count
315  Documentary       HARDLY ROBBERS             4
525      Foreign          MIXED DOORS             4
656       Horror          TRAIN BUNCH             4
593        Games         PRIVATE DROP             5
254       Comedy    FREEDOM CLEOPATRA             5
..           ...                  ...           ...
93     Animation       JUGGLER HARDLY            32
572        Games       GRIT CLOCKWORK            32
757          New  RIDGEMONT SUBMARINE            32
536      Foreign     ROCKETEER MOTHER            33
909       Travel   BUCKET BROTHERHOOD            34

[958 rows x 3 columns]
```

68.Hangi mağazalarda en çok film kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, COUNT(r.rental_id) AS rental_count
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
GROUP BY s.store_id
ORDER BY rental_count DESC;
```

	store_id	rental_count
1	1	8040
2	2	8004

B. Pandas Sorgusu ve Sonucu

```python
# Soru 68
s_cols = store_df[['store_id']]
st_cols = staff_df[['staff_id', 'store_id']]
r_cols = rental_df[['rental_id', 'staff_id']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
final_df = pd.merge(m1, r_cols, on='staff_id')

res_68 = final_df.groupby('store_id')['rental_id'].count().reset_index(name='rental_count')
res_68 = res_68.sort_values('rental_count', ascending=False)

print(res_68)
```

```
   store_id  rental_count
0         1          8040
1         2          8004
```

69.Hangi mağazalarda en az film kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, COUNT(r.rental_id) AS rental_count
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
GROUP BY s.store_id
ORDER BY rental_count ASC;
```

	store_id	rental_count
1	2	8004
2	1	8040

B. Pandas Sorgusu ve Sonucu

```python
# Soru 69
res_69 = res_68.sort_values('rental_count', ascending=True)
print(res_69)
```

```
   store_id  rental_count
1         2          8004
0         1          8040
```

70.Hangi aktörler en çok filmde rol almış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT a.first_name, a.last_name, COUNT(fa.film_id) AS film_count
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.first_name, a.last_name
ORDER BY film_count DESC;
```

	first_name	last_name	film_count
1	SUSAN	DAVIS	54
2	GINA	DEGENERES	42
3	WALTER	TORN	41
4	MARY	KEITEL	40
5	MATTHEW	CARREY	39
6	SANDRA	KILMER	37
7	SCARLETT	DAMON	36
8	ANGELA	WITHERSPOON	35
9	GROUCHO	DUNST	35
10	HENRY	BERRY	35
11	UMA	WOOD	35
12	VAL	BOLGER	35
13	VIVIEN	BASINGER	35
14	ANGELA	HUDSON	34
15	JAYNE	NOLTE	34
16	KIRSTEN	AKROYD	34
17	SIDNEY	CROWE	34

B. Pandas Sorgusu ve Sonucu

```python
# Soru 70
a_cols = actor_df[['actor_id', 'first_name', 'last_name']]
fa_cols = film_actor_df[['actor_id', 'film_id']]

merged_df = pd.merge(a_cols, fa_cols, on='actor_id')

res_70 = merged_df.groupby(['first_name', 'last_name'])['film_id'].count().reset_index(name='film_count')
res_70 = res_70.sort_values('film_count', ascending=False)

print(res_70)
```

```
    first_name  last_name  film_count
180      SUSAN      DAVIS          54
65        GINA  DEGENERES          42
190     WALTER       TORN          41
123       MARY     KEITEL          40
125    MATTHEW     CARREY          39
..         ...        ...         ...
177      SISSY   SOBIESKI          18
103      JULIA  ZELLWEGER          16
101      JULIA    FAWCETT          15
99        JUDY       DEAN          15
51       EMILY        DEE          14

[199 rows x 3 columns]
```

71.Hangi aktörler en az filmde rol almış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT a.first_name, a.last_name, COUNT(fa.film_id) AS film_count
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.first_name, a.last_name
ORDER BY film_count ASC;
```

	first_name	last_name	film_count
1	EMILY	DEE	14
2	JUDY	DEAN	15
3	JULIA	FAWCETT	15
4	JULIA	ZELLWEGER	16
5	ADAM	GRANT	18
6	SISSY	SOBIESKI	18
7	CAMERON	WRAY	19
8	PENELOPE	GUINESS	19
9	RUSSELL	CLOSE	19
10	SANDRA	PECK	19
11	BETTE	NICHOLSON	20
12	CHRIS	DEPP	20
13	CHRISTOPHER	BERRY	20
14	FAY	KILMER	20
15	KENNETH	PESCI	20
16	MATTHEW	JOHANSSON	20
17	MINNIE	KILMER	20

B. Pandas Sorgusu ve Sonucu

```python
# Soru 71
res_71 = res_70.sort_values('film_count', ascending=True)
print(res_71)
```

```
    first_name  last_name  film_count
51       EMILY        DEE          14
99        JUDY       DEAN          15
101      JULIA    FAWCETT          15
103      JULIA  ZELLWEGER          16
177      SISSY   SOBIESKI          18
..         ...        ...         ...
125    MATTHEW     CARREY          39
123       MARY     KEITEL          40
190     WALTER       TORN          41
65        GINA  DEGENERES          42
180      SUSAN      DAVIS          54

[199 rows x 3 columns]
```

72.Hangi aktörlerin oynadığı filmler en çok kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT a.first_name, a.last_name, COUNT(r.rental_id) AS rental_count
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY a.first_name, a.last_name
ORDER BY rental_count DESC;
```

	first_name	last_name	rental_count
1	SUSAN	DAVIS	825
2	GINA	DEGENERES	753
3	MATTHEW	CARREY	678
4	MARY	KEITEL	674
5	ANGELA	WITHERSPOON	654
6	WALTER	TORN	640
7	HENRY	BERRY	612
8	JAYNE	NOLTE	611
9	VAL	BOLGER	605
10	SANDRA	KILMER	604
11	SEAN	GUINESS	599
12	ANGELA	HUDSON	574
13	SCARLETT	DAMON	572
14	EWAN	GOODING	571
15	KEVIN	GARLAND	565
16	WARREN	NOLTE	564
17	CAMERON	ZELLWEGER	560

B. Pandas Sorgusu ve Sonucu

```python
# Soru 72
a_cols = actor_df[['actor_id', 'first_name', 'last_name']]
fa_cols = film_actor_df[['actor_id', 'film_id']]
f_cols = film_df[['film_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(a_cols, fa_cols, on='actor_id')
m2 = pd.merge(m1, f_cols, on='film_id')
m3 = pd.merge(m2, i_cols, on='film_id')
final_df = pd.merge(m3, r_cols, on='inventory_id')

res_72 = final_df.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index(name='rental_count')
res_72 = res_72.sort_values('rental_count', ascending=False)

print(res_72)
```

```
    first_name    last_name  rental_count
180      SUSAN        DAVIS           825
65        GINA    DEGENERES           753
125    MATTHEW       CARREY           678
123       MARY       KEITEL           674
8       ANGELA  WITHERSPOON           654
..         ...          ...           ...
101      JULIA      FAWCETT           255
99        JUDY         DEAN           255
177      SISSY     SOBIESKI           235
103      JULIA    ZELLWEGER           221
51       EMILY          DEE           216

[199 rows x 3 columns]
```

73.Hangi aktörlerin oynadığı filmler en az kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT a.first_name, a.last_name, COUNT(r.rental_id) AS rental_count
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY a.first_name, a.last_name
ORDER BY rental_count ASC;
```

	first_name	last_name	rental_count
1	EMILY	DEE	216
2	JULIA	ZELLWEGER	221
3	SISSY	SOBIESKI	235
4	JUDY	DEAN	255
5	JULIA	FAWCETT	255
6	JENNIFER	DAVIS	274
7	BETTE	NICHOLSON	279
8	ADAM	GRANT	281
9	CHRIS	DEPP	283
10	SANDRA	PECK	287
11	DAN	STREEP	289
12	MINNIE	KILMER	291
13	KENNETH	PESCI	292
14	RITA	REYNOLDS	294
15	RUSSELL	CLOSE	296
16	CAMERON	WRAY	303
17	PENELOPE	GUINESS	305

B. Pandas Sorgusu ve Sonucu

```python
# Soru 73
a_cols = actor_df[['actor_id', 'first_name', 'last_name']]
fa_cols = film_actor_df[['actor_id', 'film_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(a_cols, fa_cols, on='actor_id')
m2 = pd.merge(m1, i_cols, on='film_id')
final_df = pd.merge(m2, r_cols, on='inventory_id')

res_73 = final_df.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index(name='rental_count')
res_73 = res_73.sort_values('rental_count', ascending=True)

print(res_73)
```

```
    first_name    last_name  rental_count
51       EMILY          DEE           216
103      JULIA    ZELLWEGER           221
177      SISSY     SOBIESKI           235
99        JUDY         DEAN           255
101      JULIA      FAWCETT           255
..         ...          ...           ...
8       ANGELA  WITHERSPOON           654
123       MARY       KEITEL           674
125    MATTHEW       CARREY           678
65        GINA    DEGENERES           753
180      SUSAN        DAVIS           825

[199 rows x 3 columns]
```

74.Hangi kategorilerde en fazla aktör oynamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, COUNT(DISTINCT fa.actor_id) AS actor_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN film_actor fa ON f.film_id = fa.film_id
GROUP BY genre
ORDER BY actor_count DESC;
```

	genre	actor_count
1	Sports	182
2	Foreign	175
3	New	169
4	Documentary	168
5	Sci-Fi	167
6	Travel	166
7	Animation	166
8	Action	166
9	Family	164
10	Children	163
11	Drama	162
12	Classics	162
13	Horror	156
14	Games	150
15	Comedy	147
16	Music	144

B. Pandas Sorgusu ve Sonucu

```python
# Soru 74
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
fa_cols = film_actor_df[['film_id', 'actor_id']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
final_df = pd.merge(m1, fa_cols, on='film_id')

res_74 = final_df.groupby('name')['actor_id'].nunique().reset_index(name='actor_count')
res_74 = res_74.sort_values('actor_count', ascending=False)

print(res_74)
```

```
          name  actor_count
14      Sports          182
8      Foreign          175
12         New          169
5   Documentary          168
13      Sci-Fi          167
0       Action          166
1    Animation          166
15      Travel          166
7       Family          164
2     Children          163
3     Classics          162
6        Drama          162
10      Horror          156
9        Games          150
4       Comedy          147
11       Music          144
```

75.Hangi kategorilerde en az aktör oynamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, COUNT(DISTINCT fa.actor_id) AS actor_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN film_actor fa ON f.film_id = fa.film_id
GROUP BY genre
ORDER BY actor_count ASC;
```

	genre	actor_count
1	Music	144
2	Comedy	147
3	Games	150
4	Horror	156
5	Classics	162
6	Drama	162
7	Children	163
8	Family	164
9	Action	166
10	Animation	166
11	Travel	166
12	Sci-Fi	167
13	Documentary	168
14	New	169
15	Foreign	175
16	Sports	182

B. Pandas Sorgusu ve Sonucu

```python
# Soru 75
res_75 = res_74.sort_values('actor_count', ascending=True)
print(res_75)
```

```
           name  actor_count
11        Music          144
4        Comedy          147
9         Games          150
10       Horror          156
3      Classics          162
6         Drama          162
2      Children          163
7        Family          164
0        Action          166
1     Animation          166
15       Travel          166
13       Sci-Fi          167
5   Documentary          168
12          New          169
8       Foreign          175
14       Sports          182
```

76.Hangi kategorilerde oynayan filmler en çok kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY genre
ORDER BY rental_count DESC;
```

	genre	rental_count
1	Sports	1179
2	Animation	1166
3	Action	1112
4	Sci-Fi	1101
5	Family	1096
6	Drama	1060
7	Documentary	1050
8	Foreign	1033
9	Games	969
10	Children	945
11	Comedy	941
12	New	940
13	Classics	939
14	Horror	846
15	Travel	837
16	Music	830

B. Pandas Sorgusu ve Sonucu

```python
c_cols = category_df[['category_id', 'name']]
fc_cols = film_category_df[['category_id', 'film_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(c_cols, fc_cols, on='category_id')
m2 = pd.merge(m1, i_cols, on='film_id')
final_df = pd.merge(m2, r_cols, on='inventory_id')

res_76 = final_df.groupby('name')['rental_id'].count().reset_index(name='rental_count')
res_76 = res_76.sort_values('rental_count', ascending=False)

print(res_76)
```

```
          name  rental_count
14      Sports          1179
1    Animation          1166
0       Action          1112
13      Sci-Fi          1101
7       Family          1096
6        Drama          1060
5  Documentary          1050
8      Foreign          1033
9        Games           969
2     Children           945
4       Comedy           941
12         New           940
3     Classics           939
10      Horror           846
15      Travel           837
11       Music           830
```

77.Hangi kategorilerde oynayan filmler en az kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY genre
ORDER BY rental_count ASC;
```

	genre	rental_count
1	Music	830
2	Travel	837
3	Horror	846
4	Classics	939
5	New	940
6	Comedy	941
7	Children	945
8	Games	969
9	Foreign	1033
10	Documentary	1050
11	Drama	1060
12	Family	1096
13	Sci-Fi	1101
14	Action	1112
15	Animation	1166
16	Sports	1179

B. Pandas Sorgusu ve Sonucu

```python
# Soru 77
res_77 = res_76.sort_values('rental_count', ascending=True)
print(res_77)
```

```
          name  rental_count
11       Music           830
15      Travel           837
10      Horror           846
3     Classics           939
12         New           940
4       Comedy           941
2     Children           945
9        Games           969
8      Foreign          1033
5  Documentary          1050
6        Drama          1060
7       Family          1096
13      Sci-Fi          1101
0       Action          1112
1    Animation          1166
14      Sports          1179
```

78.En fazla kazanç sağlayan kategoriler hangileri?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, SUM(p.amount) AS total_revenue
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY genre
ORDER BY total_revenue DESC;
```

	genre	total_revenue
1	Sports	5314.21
2	Sci-Fi	4756.98
3	Animation	4656.3
4	Drama	4587.39
5	Comedy	4383.58
6	Action	4375.85
7	New	4351.62
8	Games	4281.33
9	Foreign	4270.67
10	Family	4226.07
11	Documentary	4217.52
12	Horror	3722.54
13	Children	3655.55
14	Classics	3639.59
15	Travel	3549.64
16	Music	3417.72

B. Pandas Sorgusu ve Sonucu

```python
# Soru 78
p_cols = payment_df[['rental_id', 'amount']]

full_df = pd.merge(final_df, p_cols, on='rental_id')

res_78 = full_df.groupby('name')['amount'].sum().reset_index(name='total_revenue')
res_78 = res_78.sort_values('total_revenue', ascending=False)

print(res_78)
```

```
          name  total_revenue
14      Sports        5314.21
13      Sci-Fi        4756.98
1    Animation        4656.30
6        Drama        4587.39
4       Comedy        4383.58
0       Action        4375.85
12         New        4351.62
9        Games        4281.33
8      Foreign        4270.67
7       Family        4226.07
5  Documentary        4217.52
10      Horror        3722.54
2     Children        3655.55
3     Classics        3639.59
15      Travel        3549.64
11       Music        3417.72
```

79.En az kazanç sağlayan kategoriler hangileri?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.name AS genre, SUM(p.amount) AS total_revenue
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY genre
ORDER BY total_revenue ASC;
```

	genre	total_revenue
1	Music	3417.72
2	Travel	3549.64
3	Classics	3639.59
4	Children	3655.55
5	Horror	3722.54
6	Documentary	4217.52
7	Family	4226.07
8	Foreign	4270.67
9	Games	4281.33
10	New	4351.62
11	Action	4375.85
12	Comedy	4383.58
13	Drama	4587.39
14	Animation	4656.3
15	Sci-Fi	4756.98
16	Sports	5314.21

B. Pandas Sorgusu ve Sonucu

```python
# Soru 79
res_79 = res_78.sort_values('total_revenue', ascending=True)
print(res_79)
```

```
          name  total_revenue
11       Music        3417.72
15      Travel        3549.64
3     Classics        3639.59
2     Children        3655.55
10      Horror        3722.54
5   Documentary        4217.52
7       Family        4226.07
8      Foreign        4270.67
9        Games        4281.33
12         New        4351.62
0       Action        4375.85
4       Comedy        4383.58
6        Drama        4587.39
1    Animation        4656.30
13      Sci-Fi        4756.98
14      Sports        5314.21
```

80.En çok film kiralayan müşterilerin ortalama ödeme miktarı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, AVG(p.amount) AS avg_payment
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY COUNT(r.rental_id) DESC
LIMIT 10;
```

	first_name	last_name	avg_payment
1	ELEANOR	HUNT	4.70739130434783
2	KARL	SEAL	4.92333333333333
3	CLARA	SHAW	4.65666666666667
4	MARCIA	DEAN	4.18047619047619
5	TAMMY	SANDERS	3.79487804878049
6	SUE	PETERS	3.865
7	WESLEY	BULL	4.44
8	MARION	SNYDER	4.99
9	RHONDA	KENNEDY	4.99
10	TIM	CARY	4.50282051282051

B. Pandas Sorgusu ve Sonucu

```python
# Soru 80
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, p_cols, on='rental_id')

stats = final_df.groupby(['first_name', 'last_name']).agg(
    rental_count=('rental_id', 'count'),
    avg_payment=('amount', 'mean')
).reset_index()

res_80 = stats.sort_values('rental_count', ascending=False).head(10)

print(res_80[['first_name', 'last_name', 'avg_payment']])
```

```
    first_name last_name  avg_payment
175   ELEANOR      HUNT     4.707391
318      KARL      SEAL     4.923333
379    MARCIA      DEAN     4.180476
105     CLARA      SHAW     4.656667
536     TAMMY   SANDERS     3.794878
590    WESLEY      BULL     4.440000
531       SUE    PETERS     3.865000
389    MARION    SNYDER     4.990000
551       TIM      CARY     4.502821
474    RHONDA   KENNEDY     4.990000
```

81.En az film kiralayan müşterilerin ortalama ödeme miktarı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, AVG(p.amount) AS avg_payment
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY COUNT(r.rental_id) ASC
LIMIT 10;
```

	first_name	last_name	avg_payment
1	BRIAN	WYMAN	4.40666666666667
2	KATHERINE	RIVERA	4.20428571428571
3	LEONA	OBRIEN	3.63285714285714
4	TIFFANY	JORDAN	4.27571428571429
5	ANITA	MORALES	4.19
6	CAROLINE	BOWMAN	3.39
7	ANTONIO	MEEK	4.9275
8	JEROME	KENYON	4.615
9	JOANN	GARDNER	4.1775
10	LESTER	KRAUS	4.115

B. Pandas Sorgusu ve Sonucu

```python
#Soru 81
res_81 = stats.sort_values('rental_count', ascending=True).head(10)
print(res_81[['first_name', 'last_name', 'avg_payment']])
```

```
    first_name last_name  avg_payment
71       BRIAN     WYMAN     4.406667
550    TIFFANY    JORDAN     4.275714
351      LEONA    OBRIEN     3.632857
319  KATHERINE    RIVERA     4.204286
83    CAROLINE    BOWMAN     3.390000
28       ANITA   MORALES     4.190000
277     JEROME    KENYON     4.615000
356     LESTER     KRAUS     4.115000
290      JOANN   GARDNER     4.177500
35     ANTONIO      MEEK     4.927500
```

82.Hangi mağazalarda hangi kategoriler en çok kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, c.name AS genre, COUNT(r.rental_id) AS rental_count
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY s.store_id, genre
ORDER BY rental_count DESC;
```

	store_id	genre	rental_count
1	2	Sports	614
2	2	Animation	584
3	1	Animation	582
4	1	Family	565
5	1	Sports	565
6	1	Action	562
7	1	Sci-Fi	553
8	2	Action	550
9	2	Sci-Fi	548
10	2	Foreign	543
11	2	Drama	532
12	2	Family	531
13	1	Drama	528
14	1	Documentary	525
15	2	Documentary	525
16	1	Games	494

B. Pandas Sorgusu ve Sonucu

```python
# Soru 82
s_cols = store_df[['store_id']]
st_cols = staff_df[['staff_id', 'store_id']]
r_cols = rental_df[['rental_id', 'staff_id', 'inventory_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]
fc_cols = film_category_df[['film_id', 'category_id']]
c_cols = category_df[['category_id', 'name']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
m2 = pd.merge(m1, r_cols, on='staff_id')

m3 = pd.merge(m2, i_cols, on='inventory_id')
m4 = pd.merge(m3, fc_cols, on='film_id')
final_df = pd.merge(m4, c_cols, on='category_id')

res_82 = final_df.groupby(['store_id', 'name'])['rental_id'].count().reset_index(name='rental_count')
res_82 = res_82.sort_values('rental_count', ascending=False)

print(res_82)
```

```
    store_id         name  rental_count
30         2       Sports           614
17         2    Animation           584
1          1    Animation           582
7          1       Family           565
14         1       Sports           565
0          1       Action           562
13         1       Sci-Fi           553
16         2       Action           550
29         2       Sci-Fi           548
24         2      Foreign           543
22         2        Drama           532
23         2       Family           531
6          1        Drama           528
5          1  Documentary           525
```

83.Hangi mağazalarda hangi kategoriler en az kiralanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, c.name AS genre, COUNT(r.rental_id) AS rental_count
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY s.store_id, genre
ORDER BY rental_count asc;
```

	store_id	genre	rental_count
1	2	Music	398
2	1	Travel	399
3	1	Horror	423
4	2	Horror	423
5	1	Music	432
6	2	Travel	438
7	2	New	451
8	2	Children	462
9	2	Classics	464
10	2	Comedy	466
11	1	Classics	475
12	1	Comedy	475
13	2	Games	475
14	1	Children	483
15	1	New	489
16	1	Foreign	490

B. Pandas Sorgusu ve Sonucu

```python
# Soru 83
res_83 = res_82.sort_values('rental_count', ascending=True)
print(res_83)
```

	store_id	name	rental_count
27	2	Music	398
15	1	Travel	399
10	1	Horror	423
26	2	Horror	423
11	1	Music	432
31	2	Travel	438
28	2	New	451
18	2	Children	462
19	2	Classics	464
20	2	Comedy	466
3	1	Classics	475
4	1	Comedy	475
25	2	Games	475
2	1	Children	483
12	1	New	489
8	1	Foreign	490
9	1	Games	494
5	1	Documentary	525
21	2	Documentary	525
6	1	Drama	528
23	2	Family	531
22	2	Drama	532
24	2	Foreign	543
29	2	Sci-Fi	548
16	2	Action	550
13	1	Sci-Fi	553
0	1	Action	562

84.En fazla sayıda film kiralayan mağaza hangisidir ve toplam kiralama sayısı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, COUNT(r.rental_id) AS total_rentals
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
GROUP BY s.store_id
ORDER BY total_rentals DESC
LIMIT 1;
```

	store_id	total_rentals
1	1	8040

B. Pandas Sorgusu ve Sonucu

```python
# Soru 84
s_cols = store_df[['store_id']]
st_cols = staff_df[['staff_id', 'store_id']]
r_cols = rental_df[['rental_id', 'staff_id']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
final_df = pd.merge(m1, r_cols, on='staff_id')

res_84 = final_df.groupby('store_id')['rental_id'].count().reset_index(name='total_rentals')
res_84 = res_84.sort_values('total_rentals', ascending=False).head(1)

print(res_84)
```

```
   store_id  total_rentals
0         1           8040
```

85.En az sayıda film kiralayan mağaza hangisidir ve toplam kiralama sayısı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, COUNT(r.rental_id) AS total_rentals
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
GROUP BY s.store_id
ORDER BY total_rentals ASC
LIMIT 1;
```

	store_id	total_rentals
1	2	8004

B. Pandas Sorgusu ve Sonucu

```python
# Soru 85
s_cols = store_df[['store_id']]
st_cols = staff_df[['staff_id', 'store_id']]
r_cols = rental_df[['rental_id', 'staff_id']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
final_df = pd.merge(m1, r_cols, on='staff_id')

res_85 = final_df.groupby('store_id')['rental_id'].count().reset_index(name='total_rentals')
res_85 = res_85.sort_values('total_rentals', ascending=True).head(1)

print(res_85)
```

```
   store_id  total_rentals
1         2           8004
```

86.En fazla sayıda kiralama yapan müşteri hangisidir ve toplam kiralama sayısı nedir?

C. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, COUNT(r.rental_id) AS total_rentals
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.first_name, c.last_name
ORDER BY total_rentals DESC
LIMIT 1;
```

	first_name	last_name	total_rentals
1	ELEANOR	HUNT	46

D. Pandas Sorgusu ve Sonucu

```python
# Soru 86
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]

final_df = pd.merge(c_cols, r_cols, on='customer_id')

res_86 = final_df.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index(name='total_rentals')
res_86 = res_86.sort_values('total_rentals', ascending=False).head(1)

print(res_86)
```

```
    first_name last_name  total_rentals
175    ELEANOR      HUNT             46
```

87.En az sayıda kiralama yapan müşteri hangisidir ve toplam kiralama sayısı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, COUNT(r.rental_id) AS total_rentals
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.first_name, c.last_name
ORDER BY total_rentals ASC
LIMIT 1;
```

	first_name	last_name	total_rentals
1	BRIAN	WYMAN	12

B. Pandas Sorgusu ve Sonucu

```python
#Soru 87
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]

final_df = pd.merge(c_cols, r_cols, on='customer_id')

res_87 = final_df.groupby(['first_name', 'last_name'])['rental_id'].count().reset_index(name='total_rentals')
res_87 = res_87.sort_values('total_rentals', ascending=True).head(1)

print(res_87)
```

```
   first_name last_name  total_rentals
71      BRIAN     WYMAN             12
```

88.En fazla sayıda kiralanan film hangisidir ve toplam kiralama sayısı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, COUNT(r.rental_id) AS total_rentals
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY total_rentals DESC
LIMIT 1;
```

	title	total_rentals
1	BUCKET BROTHERHOOD	34

B. Pandas Sorgusu ve Sonucu

```python
# Soru 88
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
final_df = pd.merge(m1, r_cols, on='inventory_id')

res_88 = final_df.groupby('title')['rental_id'].count().reset_index(name='total_rentals')
res_88 = res_88.sort_values('total_rentals', ascending=False).head(1)

print(res_88)
```

```
                title  total_rentals
96  BUCKET BROTHERHOOD             34
```

89.En az sayıda kiralanan film hangisidir ve toplam kiralama sayısı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, COUNT(r.rental_id) AS total_rentals
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY total_rentals ASC
LIMIT 1;
```

	title	total_rentals
1	HARDLY ROBBERS	4

B. Pandas Sorgusu ve Sonucu

```python
# Soru 89
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['inventory_id', 'film_id']]
r_cols = rental_df[['rental_id', 'inventory_id']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
final_df = pd.merge(m1, r_cols, on='inventory_id')

res_89 = final_df.groupby('title')['rental_id'].count().reset_index(name='total_rentals')
res_89 = res_89.sort_values('total_rentals', ascending=True).head(1)

print(res_89)
```

```
          title  total_rentals
558  MIXED DOORS              4
```

90.En fazla kazanç sağlayan müşteri hangisidir ve toplam kazancı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, SUM(p.amount) AS total_revenue
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY total_revenue DESC
LIMIT 1;
```

	first_name	last_name	total_revenue
1	KARL	SEAL	221.55

B. Pandas Sorgusu ve Sonucu

```python
# Soru 90
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, p_cols, on='rental_id')

res_90 = final_df.groupby(['first_name', 'last_name'])['amount'].sum().reset_index(name='total_revenue')
res_90 = res_90.sort_values('total_revenue', ascending=False).head(1)

print(res_90)
```

```
    first_name last_name  total_revenue
318       KARL      SEAL         221.55
```

91.En az kazanç sağlayan müşteri hangisidir ve toplam kazancı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, SUM(p.amount) AS total_revenue
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY c.first_name, c.last_name
ORDER BY total_revenue ASC
LIMIT 1;
```

	first_name	last_name	total_revenue
1	CAROLINE	BOWMAN	50.85

B. Pandas Sorgusu ve Sonucu

```python
# Soru 91
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['rental_id', 'customer_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, p_cols, on='rental_id')

res_91 = final_df.groupby(['first_name', 'last_name'])['amount'].sum().reset_index(name='total_revenue')
res_91 = res_91.sort_values('total_revenue', ascending=True).head(1)

print(res_91)
```

```
   first_name last_name  total_revenue
83   CAROLINE    BOWMAN          50.85
```

92.Hangi film, hangi kategoride en çok kazanç sağlamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, c.name AS genre, SUM(p.amount) AS total_revenue
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY f.title, genre
ORDER BY total_revenue DESC
LIMIT 1;
```

	title	genre	total_revenue
1	TELEGRAPH VOYAGE	Music	231.73

B. Pandas Sorgusu ve Sonucu

```python
# Soru 92
f_cols = film_df[['film_id', 'title']]
fc_cols = film_category_df[['film_id', 'category_id']]
c_cols = category_df[['category_id', 'name']]
i_cols = inventory_df[['film_id', 'inventory_id']]
r_cols = rental_df[['inventory_id', 'rental_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(f_cols, fc_cols, on='film_id')
m2 = pd.merge(m1, c_cols, on='category_id')

m3 = pd.merge(m2, i_cols, on='film_id')
m4 = pd.merge(m3, r_cols, on='inventory_id')
final_df = pd.merge(m4, p_cols, on='rental_id')

res_92 = final_df.groupby(['title', 'name'])['amount'].sum().reset_index(name='total_revenue')
res_92 = res_92.sort_values('total_revenue', ascending=False).head(1)

print(res_92)
```

```
              title   name  total_revenue
841  TELEGRAPH VOYAGE  Music         231.73
```

93.Hangi film, hangi kategoride en az kazanç sağlamış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, c.name AS genre, SUM(p.amount) AS total_revenue
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY f.title, genre
ORDER BY total_revenue ASC
LIMIT 1;
```

	title	genre	total_revenue
1	OKLAHOMA JUMANJI	New	5.94

B. Pandas Sorgusu ve Sonucu

```python
# Soru 93
f_cols = film_df[['film_id', 'title']]
fc_cols = film_category_df[['film_id', 'category_id']]
c_cols = category_df[['category_id', 'name']]
i_cols = inventory_df[['film_id', 'inventory_id']]
r_cols = rental_df[['inventory_id', 'rental_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(f_cols, fc_cols, on='film_id')
m2 = pd.merge(m1, c_cols, on='category_id')
m3 = pd.merge(m2, i_cols, on='film_id')
m4 = pd.merge(m3, r_cols, on='inventory_id')
final_df = pd.merge(m4, p_cols, on='rental_id')

res_93 = final_df.groupby(['title', 'name'])['amount'].sum().reset_index(name='total_revenue')
res_93 = res_93.sort_values('total_revenue', ascending=True).head(1)

print(res_93)
```

```
         title    name  total_revenue
847  TEXAS WATCH  Horror           5.94
```

94.En fazla kazanç sağlayan mağaza hangisidir ve toplam kazancı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, SUM(p.amount) AS total_revenue
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY s.store_id
ORDER BY total_revenue DESC
LIMIT 1;
```

	store_id	total_revenue
1	2	33881.94

B. Pandas Sorgusu ve Sonucu

```python
# Soru 94
s_cols = store_df[['store_id']]
st_cols = staff_df[['store_id', 'staff_id']]
r_cols = rental_df[['staff_id', 'rental_id']]
p_cols = payment_df[['rental_id', 'amount']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
m2 = pd.merge(m1, r_cols, on='staff_id')
final_df = pd.merge(m2, p_cols, on='rental_id')

res_94 = final_df.groupby('store_id')['amount'].sum().reset_index(name='total_revenue')
res_94 = res_94.sort_values('total_revenue', ascending=False).head(1)

print(res_94)
```

```
   store_id  total_revenue
1         2       33881.94
```

95.En az kazanç sağlayan mağaza hangisidir ve toplam kazancı nedir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, SUM(p.amount) AS total_revenue
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY s.store_id
ORDER BY total_revenue ASC
LIMIT 1;
```

	store_id	total_revenue
1	1	33524.62

B. Pandas Sorgusu ve Sonucu

```python
# Soru 95
res_95 = final_df.groupby('store_id')['amount'].sum().reset_index(name='total_revenue')
res_95 = res_95.sort_values('total_revenue', ascending=True).head(1)

print(res_95)
```

```
   store_id  total_revenue
0         1       33524.62
```

96.En fazla sayıda farklı film kiralayan müşteri hangisidir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, COUNT(DISTINCT i.film_id) AS unique_movies
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
GROUP BY c.first_name, c.last_name
ORDER BY unique_movies DESC
LIMIT 1;
```

	first_name	last_name	unique_movies
1	ELEANOR	HUNT	46

B. Pandas Sorgusu ve Sonucu

```python
#Soru 96
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['customer_id', 'inventory_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, i_cols, on='inventory_id')

res_96 = final_df.groupby(['first_name', 'last_name'])['film_id'].nunique().reset_index(name='unique_movies')
res_96 = res_96.sort_values('unique_movies', ascending=False).head(1)

print(res_96)
```

```
    first_name last_name  unique_movies
175   ELEANOR      HUNT             46
```

97.En az sayıda farklı film kiralayan müşteri hangisidir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT c.first_name, c.last_name, COUNT(DISTINCT i.film_id) AS unique_movies
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
GROUP BY c.first_name, c.last_name
ORDER BY unique_movies ASC
LIMIT 1;
```

	first_name	last_name	unique_movies
1	BRIAN	WYMAN	12

B. Pandas Sorgusu ve Sonucu

```python
# Soru 97
c_cols = customer_df[['customer_id', 'first_name', 'last_name']]
r_cols = rental_df[['customer_id', 'inventory_id']]
i_cols = inventory_df[['inventory_id', 'film_id']]

m1 = pd.merge(c_cols, r_cols, on='customer_id')
final_df = pd.merge(m1, i_cols, on='inventory_id')

grouped_df = final_df.groupby(['first_name', 'last_name'])['film_id'].nunique().reset_index(name='unique_movies')

res_97 = grouped_df.sort_values('unique_movies', ascending=True).head(1)

print(res_97)
```

```
   first_name last_name  unique_movies
71      BRIAN     WYMAN             12
```

98.En fazla sayıda farklı müşteri tarafından kiralanan film hangisidir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, COUNT(DISTINCT r.customer_id) AS unique_customers
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY unique_customers DESC
LIMIT 1;
```

	title	unique_customers
1	BUCKET BROTHERHOOD	33

B. Pandas Sorgusu ve Sonucu

```python
# SORU 98
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['film_id', 'inventory_id']]
r_cols = rental_df[['inventory_id', 'customer_id']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
final_df = pd.merge(m1, r_cols, on='inventory_id')

res_98 = final_df.groupby('title')['customer_id'].nunique().reset_index(name='unique_customers')
res_98 = res_98.sort_values('unique_customers', ascending=False).head(1)

print(res_98)
```

```
                title  unique_customers
96  BUCKET BROTHERHOOD                33
```

99.En az sayıda farklı müşteri tarafından kiralanan film hangisidir?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT f.title, COUNT(DISTINCT r.customer_id) AS unique_customers
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY unique_customers ASC
LIMIT 1;
```

	title	unique_customers
1	HARDLY ROBBERS	4

B. Pandas Sorgusu ve Sonucu

```python
# Soru 99
f_cols = film_df[['film_id', 'title']]
i_cols = inventory_df[['film_id', 'inventory_id']]
r_cols = rental_df[['inventory_id', 'customer_id']]

m1 = pd.merge(f_cols, i_cols, on='film_id')
final_df = pd.merge(m1, r_cols, on='inventory_id')
grouped_df = final_df.groupby('title')['customer_id'].nunique().reset_index(name='unique_customers')
res_99 = grouped_df.sort_values('unique_customers', ascending=True).head(1)

print(res_99)
```

```
         title  unique_customers
866  TRAIN BUNCH                 4
```

100. Hangi mağazada, hangi kategoride en fazla kazanç sağlanmış?

A. SQLite Sorgusu ve Sonucu

```sql
SELECT s.store_id, c.name AS genre, SUM(p.amount) AS total_revenue
FROM store s
JOIN staff st ON s.store_id = st.store_id
JOIN rental r ON st.staff_id = r.staff_id
JOIN payment p ON r.rental_id = p.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY s.store_id, genre
ORDER BY total_revenue DESC
LIMIT 1;
```

	store_id	genre	total_revenue
1	2	Sports	2761.85

B. Pandas Sorgusu ve Sonucu

```python
# Soru 100
s_cols = store_df[['store_id']]
st_cols = staff_df[['store_id', 'staff_id']]
r_cols = rental_df[['staff_id', 'rental_id', 'inventory_id']]
p_cols = payment_df[['rental_id', 'amount']]
i_cols = inventory_df[['inventory_id', 'film_id']]
fc_cols = film_category_df[['film_id', 'category_id']]
c_cols = category_df[['category_id', 'name']]

m1 = pd.merge(s_cols, st_cols, on='store_id')
m2 = pd.merge(m1, r_cols, on='staff_id')
m3 = pd.merge(m2, p_cols, on='rental_id')

m4 = pd.merge(m3, i_cols, on='inventory_id')
m5 = pd.merge(m4, fc_cols, on='film_id')
final_df = pd.merge(m5, c_cols, on='category_id')

res_100 = final_df.groupby(['store_id', 'name'])['amount'].sum().reset_index(name='total_revenue')
res_100 = res_100.sort_values('total_revenue', ascending=False).head(1)

print(res_100)
```

	store_id	name	total_revenue
30	2	Sports	2761.85